Technische Hochschule Augsburg

Fakultät für Informatik

Thesis

zur Erlangung des Grades
Bachelor of Engineering in Systems Engineering

Proximitätssuche von EPLAN-Makros auf Basis des LambdaRank-Algorithmus

erstellt von Andreas Alstetter

Andreas Alstetter

Pfarrer-Kempter Str. 4
89358 Kammeltal/Behlingen
T +4915201472197
andreas.alstetter@tha.de

Matrikelnummer:
2147455

Technische Hochschule Augsburg

An der Hochschule 1
D-86161 Augsburg
T +49 821 5586-0
F +49 821 5586-3222
www.tha.de
info@tha.de

Erstprüfer	Prof. Dr. Thomas Kirchmeier
Zweitprüfer	Prof. Dr. Jürgen Grinninger
Themenvergabe am	16. Juni 2026
Eingereicht am	November 14, 2026
Geheimhaltungsvereinbarung	Nein

Inhaltsverzeichnis

1 Motivation 1
1.1 Herleitung . . . 1
1.2 Herausforderung . . . 3
1.3 Wirtschaftliche Auswirkungen . . . 4
1.4 Vision . . . 5

2 Stand der Technik 7
2.1 Grundlagen . . . 7
2.1.1 Modell . . . 7
2.1.2 Neuronale Netze . . . 7
2.1.3 Vom Vektor zum Embedding-Raum . . . 8
2.1.4 Cosine Similarity . . . 9
2.2 Modellansätze . . . 10
2.2.1 Neural Language Models . . . 10
2.2.2 Large Language Models . . . 10
2.2.3 Fine-Tuning . . . 11
2.2.4 Embedding-Modelle . . . 11
2.2.5 Sentence Transformers . . . 12
2.2.6 Triplet Loss . . . 12
2.2.7 Learning-to-Rank . . . 13
2.2.8 LambdaRank . . . 14
2.3 Validierungsansätze . . . 15
2.3.1 Recall@k . . . 15
2.3.2 Mean Reciprocal Rank . . . 16
2.3.3 Normalized Discounted Cumulative Gain . . . 17
2.4 Handlungsbedarf und Forschungsfragen . . . 18

3 Vorgehensweise 20
3.1 Methodisches Vorgehen . . . 20
3.2 Iterativer Entwicklungsprozess . . . 20

4 Datenanalyse und Trainingsvorbereitung 24
4.1 Datenbasis . . . 24
4.1.1 Artikeldaten . . . 24
4.1.2 Makrodaten . . . 25

4.1.3 Artikel-Makro-Zuordnungen . . . 26
4.2 Datenaufbereitung . . . 27
4.2.1 Entfernung unvollständiger Artikel- und Makrodaten . . . 27
4.2.2 Entfernung redundanter Artikel- und Makrodaten . . . 28
4.2.3 Entfernen nicht nutzbarer Metadaten . . . 29
4.2.4 Normaliserung von Einheiten . . . 30
4.2.5 Erstellung von Klassifikationen . . . 31
4.2.6 Ergebnis der Datenaufbereitung . . . 32
4.3 Überführung in ein Trainingsformat . . . 33
4.3.1 Artikelvektor . . . 33
4.3.2 Makrovektor . . . 34
4.3.3 Lexikalvektor . . . 35
4.4 Erstellung des Datensplits . . . 36
4.4.1 Trainings-, Validierungs- und Testdaten . . . 36
4.4.2 Konsistenzprüfung der Artikel- und Makrodaten im Datensplit . . . 36
4.4.3 Ergebnis des Datensplits . . . 37
4.4.4 Architektur der Datenaufbereitung . . . 38
4.5 Kapitelergebnis . . . 39

5 Auswahl des Modellansatzes 40
5.1 Bewertungsmatrix für Modellansätze . . . 40
5.2 Large Language Models . . . 40
5.3 Sentence Transformer . . . 40
5.4 Embedding-Modell mit Triplet Loss . . . 40
5.5 Learning-to-Rank . . . 40
5.6 Modellauswahl . . . 40
5.7 Kapitelergebnis . . . 40

6 Datenanalyse und Trainigsvorbereitung 41
6.1 Analyse der Ausgangsdaten . . . 41
6.2 Aufbereitung der Trainingsdaten . . . 41
6.3 Datensplit . . . 41
6.4 Trainingsbasis . . . 41

7 Konzeption des Empfehlungssystems 42
7.1 Gesamtarchitektur . . . 42
7.1.1 Datenfluss . . . 42
7.1.2 Komponentenmodell . . . 42
7.1.3 Datenmodell . . . 42
7.2 Kapitelergebnis . . . 42

8 Implementierung des Modells 43
8.1 Kandidatenprofile . . . 43

8.2 Feature Engineering . . . 43
8.3 Trainingsmatrix . . . 43
8.4 Hyperparameteroptimierung . . . 44
8.5 Hyperparameteroptimierung . . . 44
8.6 Training . . . 45
8.7 Kapitelergebnis . . . 46

9 Evaluierung 47
9.1 Evaluierungskonzept . . . 47
9.2 Versuchsaufbau . . . 47
9.3 Versuchsergebnisse . . . 47
9.3.1 Untersuchte Konfigurationsparameter . . . 47
9.3.2 Referenzkonfiguration A01 . . . 49
9.3.3 Ergebnisse . . . 49
9.4 Analyse . . . 50
9.4.1 Same-Family-Gewicht . . . 50
9.4.2 Hierarchische Labeldefinition . . . 50
9.4.3 Profilbreite . . . 51
9.4.4 Regularisierung . . . 51
9.4.5 Top-Konfiguration . . . 51
9.5 Kapitelergebnis . . . 52

10 Diskussion 53
10.1 Nutzenbewertung . . . 53
10.2 Wirtschaftliche Betrachtung . . . 53
10.3 Grenzen . . . 53

11 Grenzen und Ausblick 54
11.1 Grenzen des aktuellen Systems . . . 54
11.1.1 Geringer Support von Ziel-Makros . . . 54
11.1.2 Variantenauflösung innerhalb einer Makrofamilie . . . 55
11.1.3 Support-Bias häufiger Ziel-Makros . . . 55
11.2 Kritische Reflexion . . . 55
11.3 Interpretation der Ergebnisse . . . 55
11.4 Ursachenanalyse der Rankingfehler . . . 56

12 Fazit und Ausblick 58
12.1 RQ1 beantworten . . . 58
12.2 RQ2 beantworten . . . 58
12.3 RQ3 beantworten . . . 58
12.4 RQ4 beantworten . . . 58
12.5 RQ0 beantworten . . . 58

Literaturverzeichnis 62

1 Motivation

1.1 Herleitung

Die zunehmende Digitalisierung in der Industrie führt zu einer fortschreitenden Vernetzung von Maschinen, Sensoren und industriellen Systemen wie zum Beispiel Produktionsanlagen in der Automobilindustrie oder Fertigungsanlagen in der Elektronikindustrie. Im gleiche Zuge entsteht eine große Mengen an Daten sowie Datenquellen, die erfasst, gespeichert und analysiert werden müssen. Dieser Anstieg an Daten und Datenquellen sorgt dafür, dass sowohl die Komplexität als auch die Dynamik von industriellen Systemen kontinuierlich zunimmt (Organisation for Economic Co-operation and Development (OECD) 2024).

Mit der steigenden Anzahl an Datenquellen und der wachsenden Datenmenge wächst auch die Herausforderung, relevante Informationen effizient zu identifizieren, zu analysieren und in fundierte Entscheidungen zu überführen. Insbesondere in komplexen industriellen Umgebungen wie der Produktion, der Konstruktion und des Engineerings müssen Mitarbeiter eine Vielzahl an Prozessparametern und Informationen gleichzeitig berücksichtigen. Die hierfür erforderliche Informationsverarbeitung übersteigt häufig die kognitiven Kapazitäten einzelner Mitarbeiter, was zu Ineffizienzen, Fehlern und Verzögerungen in den Entscheidungsprozessen führen kann (World Economic Forum und Boston Consulting Group 2023).

Künstliche Intelligenz (KI) bietet das Potenzial, diese Herausforderungen zu bewältigen, indem sie große Datenmengen analysiert sowie Muster, Zusammenhängen und Abhängigkeiten erkennt die für einen Menschen nur schwer oder nicht in Echtzeit identifizierbar sind. Darüber hinaus kann KI dazu beitragen, Entscheidungsprozesse sowie die Optimierung industrieller Systeme durch Prognosen Handlungsempfehlungen und Automatisierungen zu unterstützen, was dazu beiträgt, die Effizienz, die Qualität und die Flexibilität industrieller Systeme zu steigern und gleichzeitig die Komplexität beherrschbarer zu machen (World Economic Forum und Accenture 2026).

Die steigende Komplexität industrieller Systeme wirkt sich nicht nur auf den Betrieb von Produktionsanlagen aus, sondern beeinflusst auch die Planung, Konstruktion und das Engineering dieser Systeme. Eine Komponete der Konstruktion industrielle Systeme dabei ist die Elektrokonstruktion, die für die Planung, den Entwurf und die Dokumentation elektrischer Systeme verantwortlich ist. In der Elektrokonstruktion gehört es zu den zentralen Aufgaben Elektroschaltpläne über Projekte hinweg zu planen, zu dokumentieren, anzupassen und wiederzuverwenden. Gleichzeitig steigen die Anforderungen an Standardisierung, Qualität und Effizienz in der Elektrokonstruktion um die steigende Komplexität industrieller Systeme zu bewältigen (EPLAN 2022).

Einen etablierten Ansatz zur Bewältigung dieser Herausforderungen stellt die Makrotechnologie in EPLAN dar. EPLAN ist eine Softwarelösung für die Elektrokonstruktion, die die Planung, den Entwurf und die Dokumentation elektrischer Systeme durch Elektroschaltpläne unterstützt. Makros ermöglichen dabei die Speicherung sowie die projektübergreifende Wiederverwendung von standardisierten Baugruppen, Funktionen und Seiten aus Elektroschaltplänen um damit die Effizienz und Konsistenz in der Elektrokonstruktion zu erhöhen. Eine vereinfachte Darstellung eines Makros für einen Motor ist in Abbildung 1.1 dargestellt. Diese Makros lassen sich nach deren Erstellung in Makrobibliotheken speichern und organisieren. (EPLAN 2022)

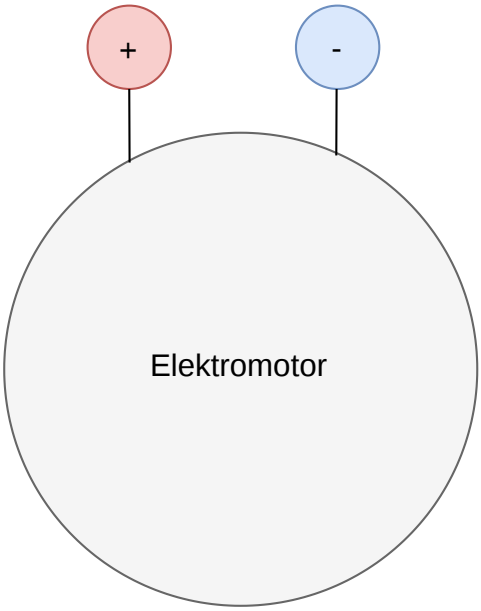


Abbildung 1.1: Elektromotor

Um ein Makro mit technischen Informationen eines Elektrobauteils wie einem Elektromotor zu verknüpfen, wird in EPLAN die Artikelverwaltung verwendet. Die Artikelverwaltung ermöglicht die Zuordnung von Artikeln zu Makros, wodurch Artikelstammdaten wie zum Beispiel Hersteller, Spannung, Strom, Leistung direkt mit einem ausgewählten Makro verknüpft werden können. Abbildung 1.2 zeigt ein Beispiel für einen Artikel eines Elektromotors, der einem Motor-Makro zugeordnet werden kann. (EPLAN 2023)



Abbildung 1.2: Beispielartikel Elektromotor

Durch diesen getrennten Aufbau ist es möglich einem Makro mehrere Artikel zuzuordnen, wodurch unterschiedliche Varianten eines Makros abgebildet werden können. Beispielhaft könnte dem Makro für einen Elektromotor ein anderer Artikel des Herstellers SEW mit 230 Volt Spannung 5 Ampere Stromstärke und einer Leistung von 1150 Watt zugeordnet werden, ohne dabei das Makro selbst ändern zu müssen. Dadurch ergibt sich eine flexible Möglichkeit, unterschiedliche Varianten eines Elektrobauteils innerhalb eines Projekts abzubilden.

1.2 Herausforderung

Mit der zunehmenden Anzahl an Makros und Artikel wächst auch die Komplexität ihrer Verwaltung und Wiederverwendung. Dies kann für eine konkrete Elektrokonstruktionsaufgabe wie der Erstellung eines Elektroschaltplans für einen Motor zu einer Herausforderung werden. Während die Wiederverwendung standardisierter Makros eine Effizienzsteigerung in der Elektrokonstruktion darstellt, steigt gleichzeitig der Aufwand das passende Makro für eine konkrete Elektrokonstruktionsaufgabe zu finden. Die Auswahl eines geeigneten Makros erfolgt dabei häufig auf Basis von Erfahrungswerten der Elektrokonstrukteure oder durch manuelle Such- und Vergleichsarbeiten.

Dies führt zu einem erhöten Zeitaufwand sowie der Nutzung suboptimaler oder redundanter Makros.

1.3 Wirtschaftliche Auswirkungen

Um die wirtschaftlichen Auswirkungen dieser Herausforderung abzuschätzen, wird hier eine beispielhafte Analyse des Zeitaufwands für die Suche und Auswahl geeigneter Makros durchgeführt und in Tabelle 1.1 zusammengefasst:

Angenommen ein Unternehmen verfügt über einen Makrobestand mit mehreren tausend Makros und ein Elektrokonstrukteur benötigt durchschnittlich fünf Minuten, um ein geeignetes Makro zu suchen, zu prüfen und auszuwählen. Werden pro Projekt beispielsweise 100 Makros wiederverwendet, ergibt sich ein Suchaufwand von rund 500 Minuten beziehungsweise 8,3 Stunden pro Projekt. Bei einem Stundensatz von 80 Euro für einen Elektrokonstrukteur entstehen für die Makrosuche Kosten in Höhe von etwa 664 Euro pro Projekt. Werden jährlich 50 vergleichbare Projekte bearbeitet, ergibt sich ein Aufwand von rund 415 Stunden was zu jährlichen Kosten von etwa 33.200 Euro führt.

Wird hingegen ein KI-basiertes Empfehlungssystem eingesetzt, das bei der Eingabe eines Datenblatts mehrere geeignete Makros vorschlägt, kann der Suchaufwand deutlich reduziert werden. Unter der Annahme, dass die benötigte Zeit für die Auswahl eines Makros von fünf Minuten auf eine Minute sinkt, reduziert sich der Gesamtaufwand auf 100 Minuten beziehungsweise 1,7 Stunden pro Projekt. Dadurch ergibt sich eine Zeitersparnis von etwa 6,6 Stunden pro Projekt. Bei einem Stundensatz von 80 Euro entspricht dies einer Kosteneinsparung von etwa 528 Euro pro Projekt beziehungsweise 26.400 Euro pro Jahr bei 50 Projekten. Zusätzlich können durch die Empfehlung bereits etablierter Makros redundante Makroneuanlagen vermieden sowie die Standardisierung und Wiederverwendung bestehender Engineering-Lösungen gefördert werden.

Den jährlichen Einsparungen müssen die Kosten für die Entwicklung und den Betrieb des Empfehlungssystems gegenübergestellt werden. Für ein prototypisches KI-basiertes Empfehlungssystem sind Entwicklungsaufwände für Datenaufbereitung, Modellentwicklung, Integration in die bestehende Engineering-Umgebung sowie Validierung und Tests erforderlich.

Angenommen, die Entwicklung des Systems verursacht einen einmaligen Aufwand von 500 Arbeitsstunden. Bei einem durchschnittlichen internen Stundensatz von 80 Euro ergeben sich Entwicklungskosten von etwa 40.000 Euro. Zusätzlich fallen jährliche Kosten für Wartung, Weiterentwicklung und den Betrieb der IT-Infrastruktur an. Werden hierfür beispielsweise 50 Stunden pro Jahr veranschlagt, entstehen laufende Kosten von rund 4.000 Euro jährlich.

Demgegenüber steht die zuvor berechnete jährliche Einsparung von etwa 26.400 Euro durch die reduzierte Suchzeit bei der Makroauswahl. Unter dieser Annahme amortisieren sich die Entwicklungskosten nach etwa 1,8 Jahren. Nach Erreichen des Break-even-Punkts führt das System zu einer jährlichen Nettoeinsparung von rund 22.400 Euro.

Zusätzlich sind mehrere wirtschaftliche Effekte in dieser vereinfachten Betrachtung noch nicht berücksichtigt. Hierzu zählen insbesondere die Vermeidung redundanter Makroneuanlagen, eine höhere Standardisierung bestehender Lösungen, ein geringerer Einarbeitungsaufwand neuer Mitarbeiter sowie eine Reduzierung von Such- und Auswahlfehlern. Die tatsächlichen wirtschaftlichen Vorteile können daher über der allein aus der Zeiteinsparung berechneten Kostenreduktion liegen.

Tabelle 1.1: Beispielhafte Wirtschaftlichkeitsbetrachtung des Empfehlungssystems

Kennzahl	Ohne KI	Mit KI
Suchzeit pro Makro	5 min	1 min
Makros pro Projekt	100	100
Gesamtzeit pro Projekt	500 min (= 8,3 h)	100 min (= 1,7 h)
Stundensatz Konstrukteur	80 €	80 €
Kosten pro Projekt	664 €	136 €
Projekte pro Jahr	50	50
Personalkosten pro Jahr	33.200 € (664 € · 50)	6.800 € (136 € · 50)
Kosten des KI-Systems		
Entwicklungskosten (einmalig)	-	40.000 €
Wartungskosten pro Jahr	-	4.000 €
Gegenüberstellung Kosten/Jahr		
Gesamtkosten pro Jahr	33.200 €	10.800 € (6.800 € + 4.000 €)
Ergebnis		
Jährliche Einsparung	33.200 € - 10.800 € = **22.400 €**	
Amortisationszeit	40.000 € / 22.400 € = **1,8 Jahre**	

1.4 Vision

Die in der Artikelverwaltung vorhandenen Zuordnungen zwischen Artikeln und Makros stellen eine wertvolle Wissenquelle dar. Dieses vorhandene Wissen eröffnet die Möglichkeit, Methoden der Künstlichen Intelligenz (KI) zur Unterstützung von Elektrokonstrukteuren einzusetzen. Die Vision 1.3 besteht darin, ein KI-basiertes Empfehlungssystem zu entwickeln, das auf die in der Artikelverwaltung vorhandenen Zuordnungen zwischen Artikeln und Makros trainiert wird, um geeignete Makros für eine konkrete Elektrokonstruktionsaufgabe vorzuschlagen. Hierbei dienen die Artikelstammdaten wie Hersteller, Spannung, Strom und Leistung als Eingabedaten, während die zu empfehlende Makros als Ausgabedaten, für das Empfehlungssystem dienen. Die Eingabedaten sollen dabei aus einem Artikel extrahiert, und für die Suchanfrage an das Empfehlungssystem verwendet werden. Die Ausgabedaten des Empfehlungssystem stellen die am besten zu den Eingabedaten passenden Makros dar. Diese Makros sollen dann entsprechend ihrer Eignung für

die konkrete Elektrokonstruktionsaufgabe in absteigender Reihenfolge in einer Rangliste sortiert werden.

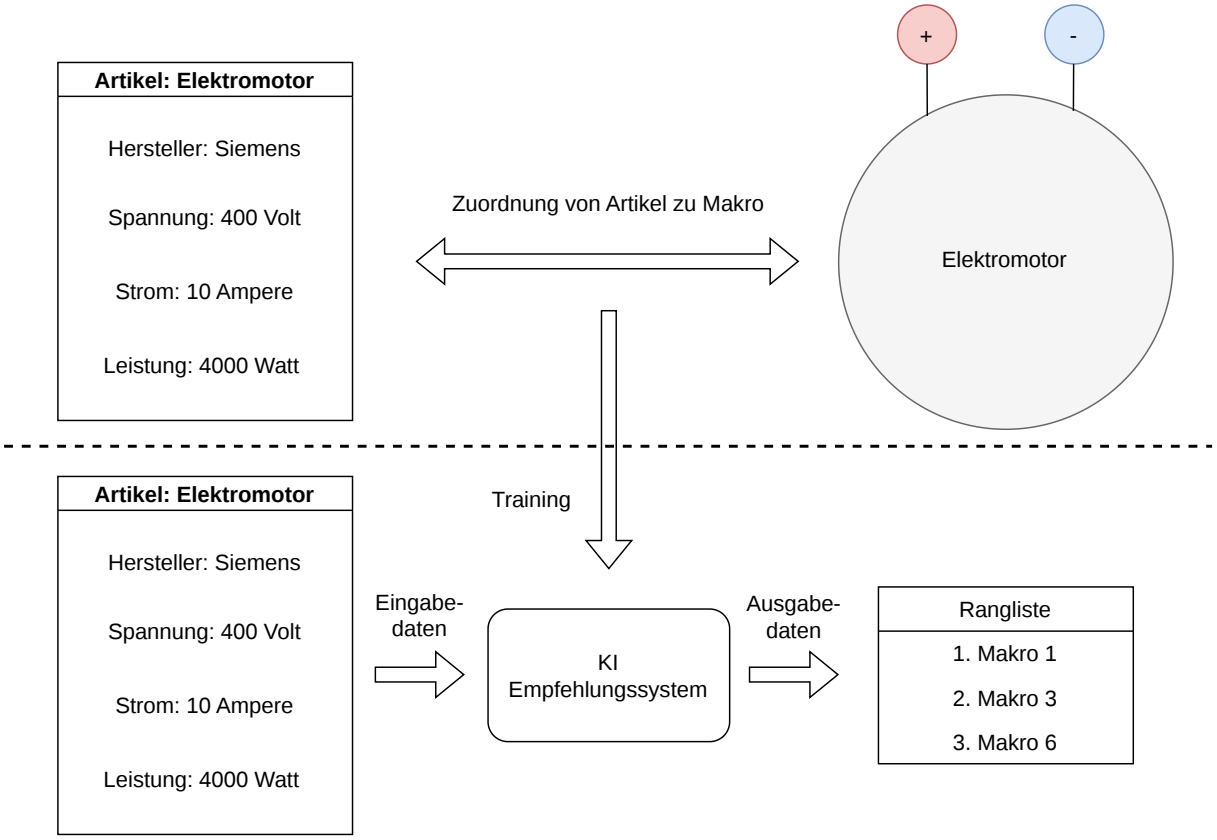


Abbildung 1.3: Vision des Empfehlungssystems

Durch die Nutzung eines solchen Empfehlungssystems soll es möglich werden, den Suchaufwand für geeignete Makros aus einem bestehenden Makrobestand zu reduzieren, und gleichzeitig einen Beitrag zur weiteren Automatisierung wissensintensiver Tätigkeiten und der Nutzung der Potenziale Künstlicher Intelligenz in der Industrie zu leisten.

2 Stand der Technik

2.1 Grundlagen

2.1.1 Modell

Ein Modell ist das Ergebnis eines Lernprozesses, bei dem aus einer großen Menge von Beispieldaten Muster und Zusammenhänge gelernt, und dabei die Parameter des Modells angepasst werden. Das dabei erlernte Wissen wird im Modell gespeichert und kann anschließend genutzt werden, um neue Eingaben zu verarbeiten und passende Vorhersagen zu erzeugen. Ein Modell bildet somit die Grundlage dafür, bereits gelernte Zusammenhänge auf bisher unbekannte Daten anzuwenden (LeCun u. a. 2015).

2.1.2 Neuronale Netze

Ein Neuronales Netz wie in Abbildung 2.1 dargestellt, ist ein Netzwerk aus miteinander verbundenen Knoten, den sogenannten Neuronen, die in ihrer Struktur und Funktionsweise an das menschliche Gehirn angelehnt sind. Diese Neuronen sind durch Verbindungen miteinander Verknüpft, die die Stärke der Signalübertragung zwischen den Neuronen bestimmen. Ein Neuronales Netz besteht aus mehreren Schichten von Neuronen wie Eingabeschicht, versteckten Schichten und Ausgabeschicht. Diese Schichten arbeiten zusammen und verarbeiten die Eingaben wie Text, Bilder oder Sensordaten schrittweise. Jede Schicht hat dabei eine spezifische Funktion: Die Eingabeschicht nimmt die Rohdaten auf, die versteckten Schichten extrahieren Merkmale und Muster, und die Ausgabeschicht liefert die endgültigen Vorhersagen für den jeweiligen Anwendungsfall. Während dem Lernens, dem sogenannten Training werden die Verbindungen und deren Gewichtungen zwischen den Neuronen schrittweise wie in Abschnitt 2.1.1 beschrieben angepasst. (LeCun u. a. 2015).

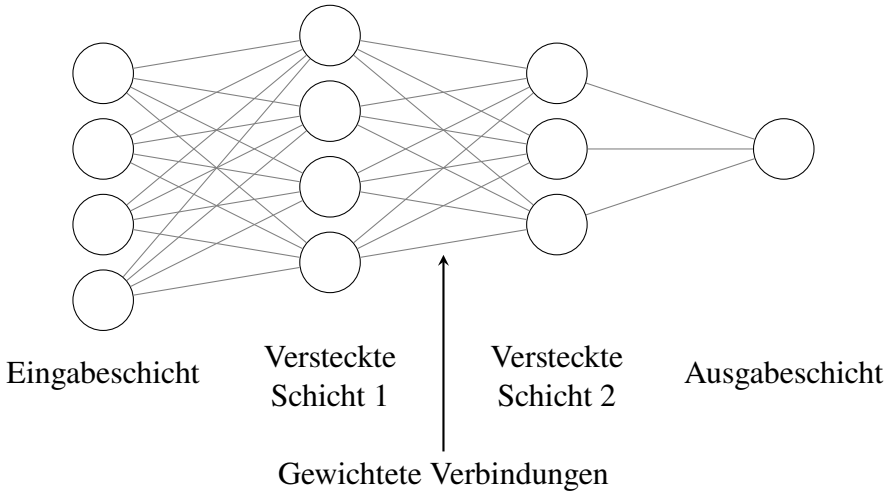


Abbildung 2.1: Schematischer Aufbau eines mehrschichtigen neuronalen Netzes

2.1.3 Vom Vektor zum Embedding-Raum

Vektoren sind geordnete Mengen von Zahlen, die in der Mathematik und Informatik verwendet werden, um Objekte im mehrdimensionalen Vektorraum darzustellen. In der Verarbeitung natürlicher Sprache werden Wörter, Sätze und Dokumente häufig in Vektoren überführt. Ein anderes Wort für diese Vektoren ist Embeddings. Embeddings ermöglichen es, semantische Ähnlichkeiten zwischen Wörtern zu erfassen, indem ähnliche Wörter im mehrdimensionalen Embedding-Raum (vergleiche Abbildung 2.2) nahe beieinander liegen. In diesem Raum repräsentieren die Positionen der Embeddings die semantischen Beziehungen zwischen den entsprechenden Wörtern. Jedes Wort wird dabei durch mehrere Zahlenwerte, welche die verschiedenen Merkmale des Wortes repräsentieren, in einem Embedding dargestellt. Die einzelnen Dimensionen eines Embeddings besitzen hier keine direkte, für Menschen verständliche Bedeutung. Erst die Position der Embeddings im Raum ermöglicht es, semantische Ähnlichkeiten zwischen Wörtern zu erkennen (Mikolov u. a. 2013).

Ein Embedding kann dabei wie folgt veranschaulicht werden:

$$\mathbf{e}_{\text{Katze}} = \begin{bmatrix} 0.82 \\ -0.15 \\ 0.47 \end{bmatrix}$$

Dieses Embedding repräsentiert das Wort „Katze“ im mehrdimensionalen Embedding-Raum, wobei jede Komponente des Vektors ein bestimmtes Merkmal oder eine Eigenschaft des Wortes „Katze“ kodiert. Um die Funktionalität von Embeddings weiter zu verdeutlichen, sollen hier mehrere Embeddings für verschiedene Wörter exemplarisch betrachtet werden:

$$\mathbf{e}_{\text{Katze}} = \begin{bmatrix} 0.82 \\ -0.15 \\ 0.47 \end{bmatrix}, \qquad \mathbf{e}_{\text{Hund}} = \begin{bmatrix} 0.79 \\ -0.12 \\ 0.51 \end{bmatrix}, \qquad \mathbf{e}_{\text{Auto}} = \begin{bmatrix} -0.34 \\ 0.88 \\ 0.70 \end{bmatrix}$$

Abbildung 2.2 zeigt die Position dieser Embeddings in einem vereinfachten dreidimensionalen Embedding-Raum. Da die Koordinaten der Wörter „Katze“ und „Hund“ sehr ähnlich sind, liegen ihre Positionen nahe beieinander. Das Embedding des Wortes „Auto“ unterscheidet sich dagegen stärker von „Katze“ oder „Hund“ und befindet sich deshalb weiter entfernt.

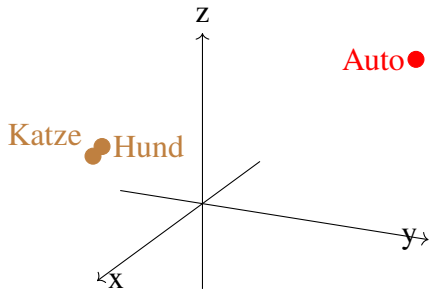


Abbildung 2.2: Vereinfachte Darstellung von Embeddings im dreidimensionalen Embedding-Raum

2.1.4 Cosine Similarity

Die Cosine Similarity vergleicht den Winkel sowie die Längen zwischen zwei Vektoren im Embedding-Raum. Je kleiner der Winkel zwischen diesen Vektoren und je ähnlicher ihre Längen sind, desto ähnlicher sind diese Vektoren im Embedding-Raum zueinander (Reimers und Gurevych 2019).

Zur Veranschaulichung soll die Cosine Similarity zunächst für die Wörter „Katze“ und „Hund“ berechnet werden.

$$\mathbf{e}_{\text{Katze}} = \begin{bmatrix} 0.82 \\ -0.15 \\ 0.47 \end{bmatrix}, \qquad \mathbf{e}_{\text{Hund}} = \begin{bmatrix} 0.79 \\ -0.12 \\ 0.51 \end{bmatrix}$$

Zunächst wird das Skalarprodukt beider Vektoren berechnet:

$$\begin{aligned} \mathbf{e}_{\text{Katze}} \cdot \mathbf{e}_{\text{Hund}} &= (0.82 \cdot 0.79) + (-0.15 \cdot -0.12) + (0.47 \cdot 0.51) \\ &= 0.6478 + 0.0180 + 0.2397 = 0.9055 \end{aligned}$$

Anschließend werden die Längen der beiden Vektoren bestimmt:

$$\|\mathbf{e}_{\text{Katze}}\| = \sqrt{0.82^2 + (-0.15)^2 + 0.47^2}$$
$$= \sqrt{0.6724 + 0.0225 + 0.2209} = \sqrt{0.9158} = 0.9570$$
$$\|\mathbf{e}_{\text{Hund}}\| = \sqrt{0.79^2 + (-0.12)^2 + 0.51^2}$$
$$= \sqrt{0.6241 + 0.0144 + 0.2601} = \sqrt{0.8986} = 0.9480$$

Damit ergibt sich die Cosine Similarity zu

$$\text{cosine_similarity}(\mathbf{e}_{\text{Katze}}, \mathbf{e}_{\text{Hund}}) = \frac{0.9055}{0.9570 \cdot 0.9480} = 0.998$$

Der errechnete Wert von 0.998 liegt sehr nahe bei 1. Dies zeigt, dass die Embeddings von „Katze“ und „Hund“ nahezu in dieselbe Richtung weisen und daher als semantisch sehr ähnlich betrachtet werden.

Wird dagegen die Cosine Similarity zwischen „Katze“ und „Auto“ berechnet, ergibt sich analog zur Berechnung für „Katze“ und „Hund“ ein Wert von -0.073. Der Wert liegt nahe bei 0 und sogar leicht im negativen Bereich. Die beiden Embeddings weisen daher in deutlich unterschiedliche Richtungen und werden als semantisch unähnlich betrachtet.

2.2 Modellansätze

2.2.1 Neural Language Models

Neuronale Sprachmodelle (Neural Language Models, NLMs) sind spezielle neuronale Netze, die auf Textsammlungen vortrainiert werden, um sprachliche Muster, semantische Zusammenhänge sowie Wissensrepräsentationen zu erlernen. Hierzu werden Wörter, Sätze und Dokumente in Embeddings überführt, die als Eingabe für ein neuronales Netz, wie in Abschnitt 2.1.2 beschrieben, dienen (LeCun u. a. 2015).

2.2.2 Large Language Models

Large Language Models (LLMs) basieren auf neuronalen Sprachmodellen und erweitern diese durch eine enorme Anzahl an Parametern sowie tieferen Architekturen. Sie bilden die Grundlage für generative KI-Systeme und ermöglichen die Bearbeitung von verschiedenen Aufgaben aus dem bereich der natürlichen Sprachverarbeitung. Dazu zählen die Generierung (Erzeugung von Texten) , die Klassifikation (Einordnung von Texten in vordefinierte Kategorien), Frage-Antwort-Systeme sowie das liefern mehrer relevanter Ergebnisse zu einer Suchanfrage. (Zhao u. a. 2026).

Die Architektur hinter einem LLM basiert auf dem Transformer-Modell. Dieses Modell nutzt sogenannte Attention-Mechanismen, um die statistischen Zusammenhänge zwischen einzelen Wörtern in einem Satz zu erfassen und zu gewichten (Vaswani u. a. 2017). Ein Beispiel hierfür ist folgender Satz: „Anna trinkt Kaffee am Morgen.“ Der Attention-Mechanismus bewertet die Bedeutung jedes Wortes im Zusammenhang mit den anderen Wörtern des Satzes. Dabei erkennt das Modell, dass die Wörter „Anna“, „trinkt“ und „Kaffee“ stärker miteinander in Beziehung stehen als häufig vorkommende Wörter wie „am“ oder „der“.

2.2.3 Fine-Tuning

Large Language Models werden auf allgemeine Textsammlungen vortrainiert, um ein umfangreiches sprachliches und sachliches Wissen zu erwerben. Dies führt dazu, dass das Modell ein breites Wissen erwirbt, aber möglicherweise nicht auf spezifische Aufgaben oder einzelnen Fachbereiche spezialisiert ist. Um ein LLM auf spezifische Aufgaben oder Fachbereiche anzupassen, wird häufig das sogenannte Fine-Tuning eingesetzt. Hierbei wird das vortrainierte LLM mit Informationen zu der spezifischen Aufgabe oder dem Fachbereich weiter trainiert, um damit die Leistungsfähigkeit an die Anforderrungen des jeweiligen Einsatzbereichs anzupassen (Wei u. a. 2022; Zhang u. a. 2025).

Ein Vorteil des Fine-Tunings ist es, dass ein vortrainiertes LLM nicht vollständig neu trainiert werden muss, sondern das vorhanden Wissen in als Ausgangspunkt genutzt werden kann und dieses um die neuen spezifischen Informationen erweitert wird. Dadurch ist es möglich, deutlich bessere Ergebnisse in kürzerer Zeit zu erzielen, als bei einem kompletten Neutraining eines neuronalen Sprachmodells (Wei u. a. 2022).

Um dies am obrigen Beispiel mit Anna zu verdeutlichen: Angenommen, das LLM soll zusätzlich Informationen über die Vorlieben von Anna beim Kaffeetrinken lernen. Anna hat eine Vorliebe für Espresso aus Italien. Durch Fine-Tuning ist es möglich, die Attention-Mechanismen des Modells gezielt auf die Präferenz von Anna auszurichten. Nach dem Fine-Tuning weiß das Modell nicht nur, dass Anna Kaffee am morgen trinkt, sondern auch, dass sie Espresso aus Italien bevorzugt.

2.2.4 Embedding-Modelle

Embedding-Modelle bauen auf dem Grundpinzip aus Kapitel **??** auf. Es handelt sich um Modelle, welche jegliche Arte von Eingaben wie Wörter, Sätze, Bilder oder Audioinhalte in Embeddings überführen und diese Eingaben dann in einem Embedding-Raum abbilden. Dadurch ist es möglich, die semantische Ähnlichkeit zwischen verschiedenen Eingaben zu bestimmen, indem die Abstände ihrer Embeddings im Embedding-Raum betrachtet werden. Hierfür wird typischerweise die Cosine Similarity (vergleiche Abschnitt 2.1.4) zwischen den Embeddings berechnet, um die semantische Ähnlichkeit zu quantifizieren.

2.2.5 Sentence Transformers

Bei Sentence Transformers handelt es sich um eine weiterentwickelte Form von Embedding-Modellen, die vollständige Sätze oder Dokumente in Embeddings, überführen. Im Gegensatz zu einem LLM, das auf die Vorhersage des nächsten Wortes trainiert wird, liegt der Fokus bei Sentence Transformers darauf, semantisch ähnliche Texte im Embedding-Raum nahe beieinander zu platzieren, während semantisch unterschiedliche Texte weiter voneinander entfernt liegen. Sentence Transformers werden häufig für Aufgaben wie semantische Suche oder Empfehlungssysteme eingesetzt. Anstatt ausschließlich identische Schlüsselwörter zu suchen, ermöglichen sie die Identifikation inhaltlich ähnlicher Dokumente auch dann, wenn unterschiedliche Formulierungen verwendet werden (Reimers und Gurevych 2019).

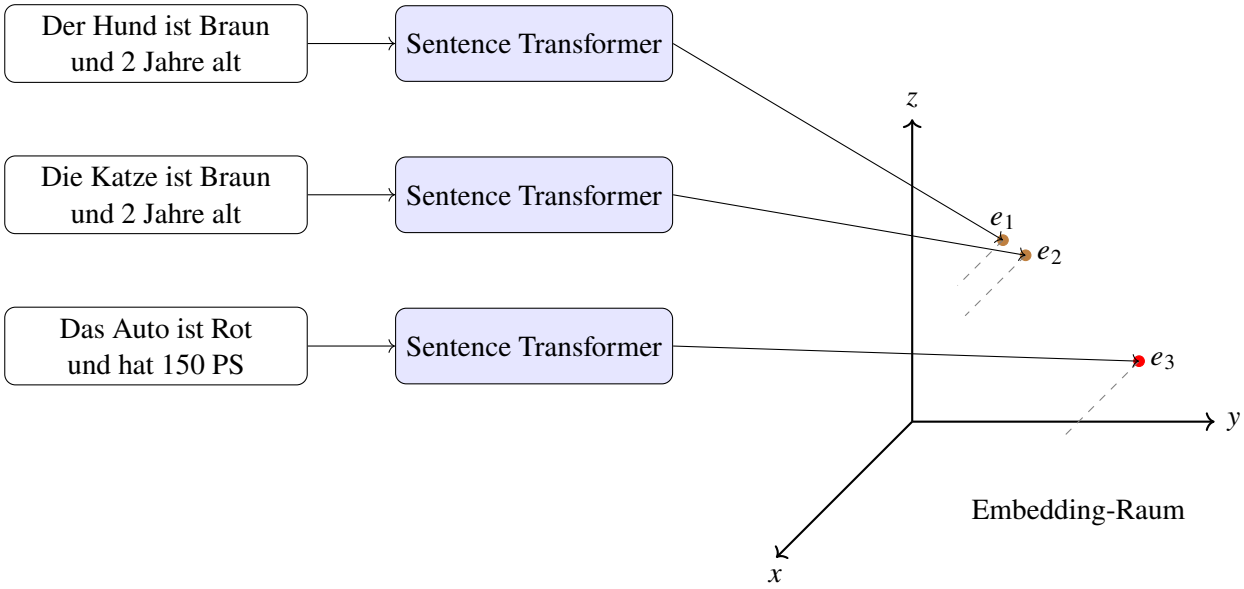


Abbildung 2.3: Prinzipielle Funktionsweise eines Sentence Transformers. Semantisch ähnliche Texte werden auf nahe beieinanderliegende Embeddings abgebildet, während inhaltlich unterschiedliche Texte größere Abstände im Embedding-Raum aufweisen.

2.2.6 Triplet Loss

Um mithilfe von Embedding-Modellen semantische Ähnlichkeiten von Daten wie Texten oder Bildern zu lernen, wird häufig der Triplet Loss (vergleiche Abbildung 2.4) eingesetzt. Hierzu werden während des Trainings Dreiergruppen von Trainingsbeispielen, sogenannte Triplets, verwendet. Jedes Triplet besteht aus einem Anker (A), einem positiven Beispiel (P) und einem negativen Beispiel (N). Zusätzlich gibt es noch den Margin-Parameter (m). Dieser legt fest, um wie viel der Abstand zwischen Anker und negativem Beispiel größer sein muss als der Abstand zwischen Anker und positivem Beispiel. Das Ziel des Triplet Loss besteht darin, den Abstand zwischen Anker und positivem Beispiel zu minimieren und gleichzeitig den Abstand zwischen Anker und

negativem Beispiel um mindestens eine vorgegebene Margin (m) zu vergrößern (Schroff u. a. 2015).

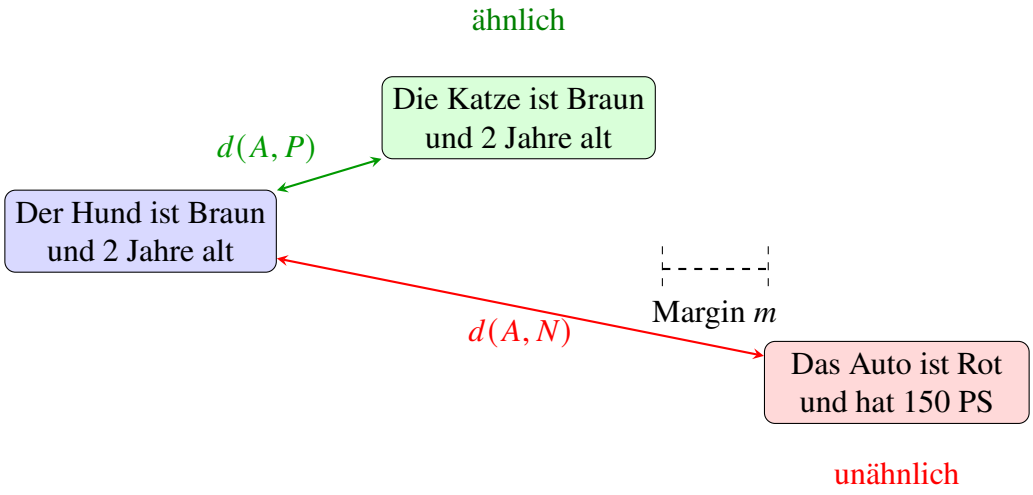


Abbildung 2.4: Funktionsweise des Triplet Loss. Der Anchor (A) wird dem positiven Beispiel (P) angenähert, während der Abstand zum negativen Beispiel (N) vergrößert wird. Die Margin m definiert dabei den Mindestabstand, um den das negative Beispiel weiter vom Anchor entfernt liegen muss als das positive Beispiel.

2.2.7 Learning-to-Rank

Learning-to-Rank (LTR) bezeichnet eine Klasse von maschinellen Lernverfahren, deren Ziel darin besteht, die Reihenfolge von Ergebnissen innerhalb einer Rangliste zu optimieren. Die Grundidee besteht darin, für eine Anfrage mehrere mögliche Ergebnisse zu bewerten und anschließend entsprechend ihrer Relevanz zu sortieren. Je relevanter ein Ergebnis für die Anfrage ist, desto weiter oben soll es in der Rangliste erscheinen (Liu 2009).

Grundsätzlich lassen sich Learning-to-Rank-Verfahren in drei Kategorien unterteilen:

Pointwise: Das Modell betrachtet jedes Ergebnis einzeln und bewertet dessen Relevanz unabhängig von den übrigen Ergebnissen.

Pairwise: Das Modell betrachtet jeweils zwei Ergebnisse gleichzeitig und lernt, welches der beiden relevanter ist.

Listwise: Das Modell betrachtet die gesamte Rangliste und optimiert direkt deren Qualität.

Die folgende Abbildung 2.5 veranschaulicht die Unterschiede zwischen den drei Kategorien von Learning-to-Rank-Verfahren.

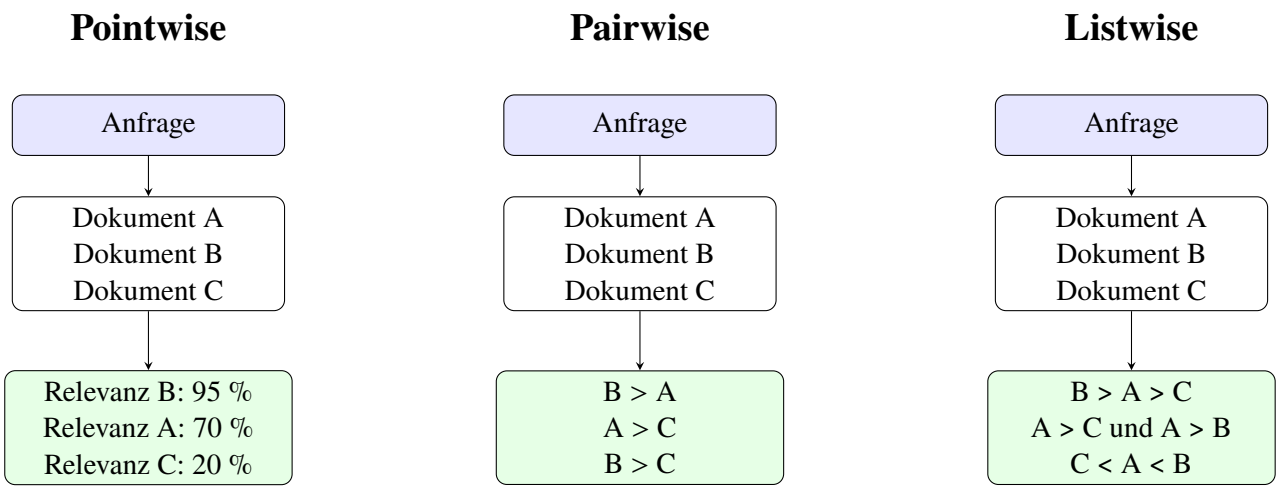


Abbildung 2.5: Vergleich der Learning-to-Rank-Ansätze. Pointwise bewertet Dokumente einzeln, Pairwise vergleicht Dokumentenpaare miteinander und Listwise optimiert die vollständige Rangliste direkt.

2.2.8 LambdaRank

LambdaRank ist eine Weiterentwicklung des zuvor beschriebenen Pairwise-Ansatzes des Learning-to-Rank-Verfahrens. Wie beim Pairwise-Ranking vergleicht das Modell jeweils zwei Ergebnisse gleichzeitig und lernt, welches Ergebnis für eine Anfrage relevanter ist. Der Unterschied besteht darin, dass bei LambdaRank nicht alle Vergleiche gleich stark gewichtet werden (Burges u. a. 2006).

Die Grundidee von LambdaRank besteht darin, die Position der Dokumente innerhalb der Rangliste während des Trainings zu berücksichtigen. Hierbei wird bewertet, wie stark sich eine Fehlplatzierung auf die Qualität der gesamten Rangliste auswirkt.

Abbildung 2.6 zeigt beispielhaft eine Rangliste, in der sich das eigentlich relevanteste Dokument auf Rang 3 befindet. Da eine Korrektur dieser Fehlplatzierung die Qualität der Rangliste deutlich verbessern würde, erhält dieser Vergleich während des Trainings eine hohe Gewichtung. Fehlplatzierungen auf den hinteren Rangpositionen, beispielsweise zwischen Rang 4 und Rang 5, haben dagegen meist nur einen geringen Einfluss auf die wahrgenommene Qualität der Rangliste und werden entsprechend schwächer gewichtet. Zur Umsetzung verwendet LambdaRank sogenannte λ-Gradienten. Ein Gradient beschreibt hierbei die Richtung und Stärke der Anpassung einer Position innerhalb der Rangliste. Dokumentenvergleiche mit großem Einfluss auf die Rangliste erzeugen große λ-Werte, während Vergleiche mit geringem Einfluss kleinere λ-Werte erzeugen. Dadurch lernt das Modell insbesondere Fehler auf den oberen Rangpositionen zu korrigieren (Burges u. a. 2006).

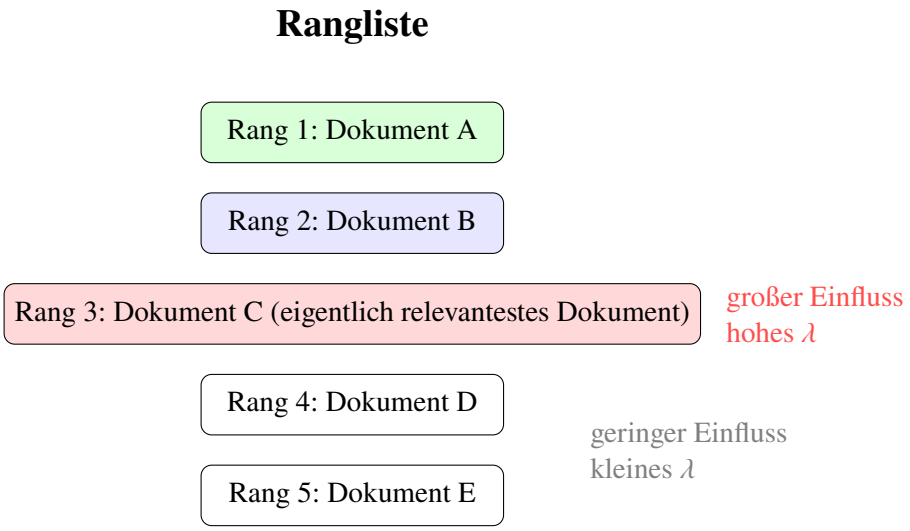


Abbildung 2.6: Prinzip von LambdaRank. Vergleiche, deren Vertauschung die Rangliste stark verändert, erhalten eine höhere Gewichtung als Vergleiche mit geringem Einfluss auf die Rangfolge (Burges u. a. 2006).

2.3 Validierungsansätze

2.3.1 Recall@k

Diese Metrik beantwortet die Frage, ob eine Anfrage an ein Empfehlungssystem ein relevantes Ergebnis innerhalb der ausgegebenen Rangliste liefert, wenn lediglich die ersten k Plätze dieser Rangliste betrachtet werden. Dabei wird nicht berücksichtigt, an welchem Platz innerhalb dieser ersten k Positionen sich das relevante Ergebnis befindet. Es wird ausschließlich geprüft, ob mindestens ein relevantes Ergebnis innerhalb der Top-k Positionen vorhanden ist.(Manning u. a. 2008).

Abbildung 2.7 veranschaulicht das Prinzip von Recall@k an einem konkreten Beispiel für $k = 3$.

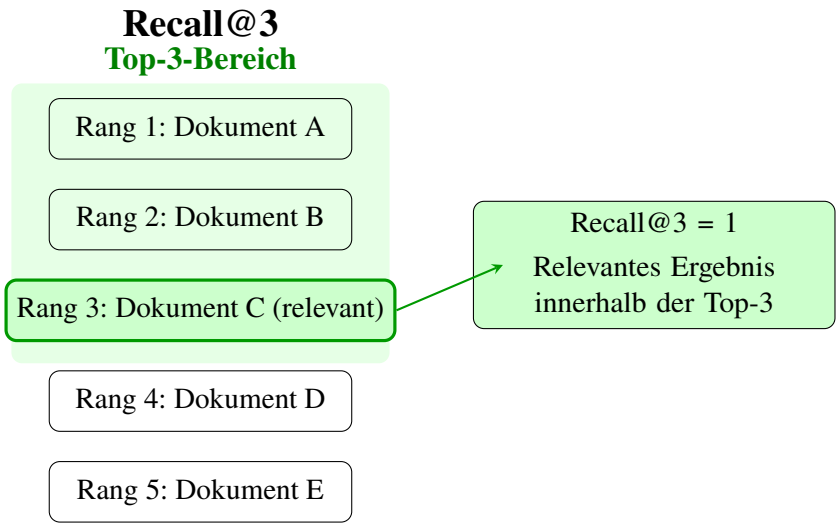


Abbildung 2.7: Beispiel für Recall@3. Da sich das relevante Dokument C innerhalb der ersten drei Rangpositionen befindet, gilt die Anfrage als erfolgreich und trägt positiv zum Recall@3-Wert bei.

Der Wertebereich der Berechnung liegt zwischen 0 und 1. Ein Wert von 1 bedeutet, dass für jede Anfrage mindestens ein relevantes Ergebnis innerhalb der ersten k Positionen gefunden wurde.

2.3.2 Mean Reciprocal Rank

Der Mean Reciprocal Rank (MRR) bewertet, wie früh das erste relevante Ergebnis einer Anfrage an das Empfehlungssystem innerhalb der ausgegebenen Rangliste erscheint (Voorhees und Tice 1999; Zhu u. a. 2024). Während Recall@k lediglich betrachtet, ob ein relevantes Ergebnis gefunden wird, berücksichtigt der MRR zusätzlich dessen Position.

Das Zentrale Element des MRR ist der Reziproke Rang. Dieser gibt an, an welcher Position sich das erste relevante Ergebnis einer Rangliste befindet. Dazu wird der Kehrwert (Reziprokwert) der Rangposition berechnet: Befindet sich das erste relevante Ergebnis auf Rang 3, beträgt der reziproke Rang $\frac{1}{3}$. Je früher das erste relevante Ergebnis erscheint, desto größer ist der reziproke Rang. Dadurch werden relevante Treffer auf den vorderen Positionen stärker belohnt als Treffer auf den hinteren Positionen. (Voorhees und Tice 1999; Zhu u. a. 2024)

Im Beispiel aus Abbildung 2.8 befindet sich das relevante Ergebnis auf Rang 3. Ein relevantes Ergebnis auf Rang 1 oder 2 würde in diesem Beispiel besser bewertet als das Ergebnis auf Rang 3.

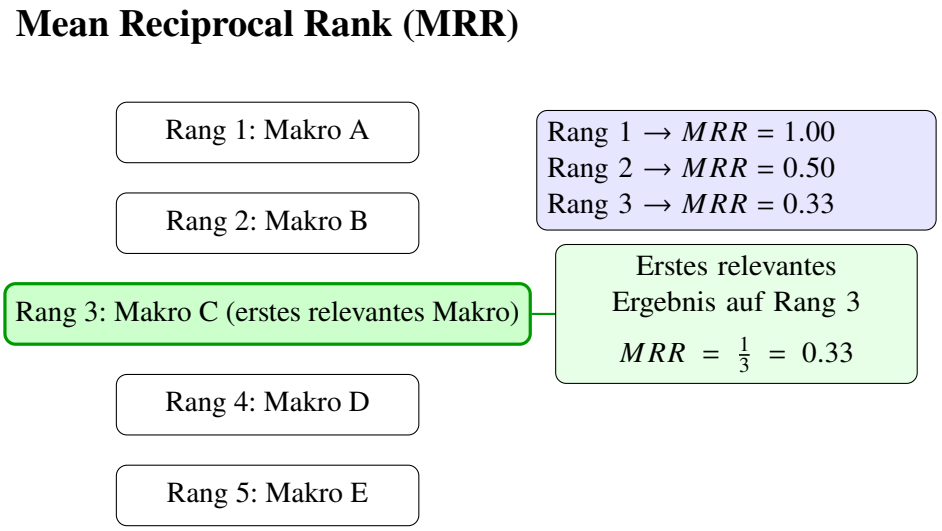


Abbildung 2.8: Beispiel zur Berechnung des Mean Reciprocal Rank. Das erste relevante Makro befindet sich auf Rang 3, wodurch sich ein reziproker Rang von $1/3$ ergibt. Frühere relevante Treffer führen zu höheren MRR-Werten.

Der Wertebereich liegt auch hier zwischen 0 und 1. Ein hoher Wert zeigt an, dass ein relevantes Ergebnis bereits an einer der vorderen Positionen einer Rangliste erscheint.

2.3.3 Normalized Discounted Cumulative Gain

Der Normalized Discounted Cumulative Gain (nDCG) bewertet die Qualität einer gesamten Rangliste. Im Gegensatz zu Recall@k und MRR wird hierbei nicht nur betrachtet, ob relevante Ergebnisse gefunden werden oder wie früh das erste relevante Ergebnis erscheint. Stattdessen werden alle Ergebnisse innerhalb der Rangliste berücksichtigt. Zusätzlich unterscheidet nDCG zwischen verschiedenen Relevanzstufen. Dadurch kann beispielsweise ein sehr relevantes Ergebnis stärker bewertet werden als ein nur teilweise relevantes Ergebnis (Järvelin und Kekäläinen 2002).

Zur Veranschaulichung wird in Abbildung 2.9 eine Rangliste mit unterschiedlichen Relevanzwerten betrachtet.

Normalized Discounted Cumulative Gain (nDCG)

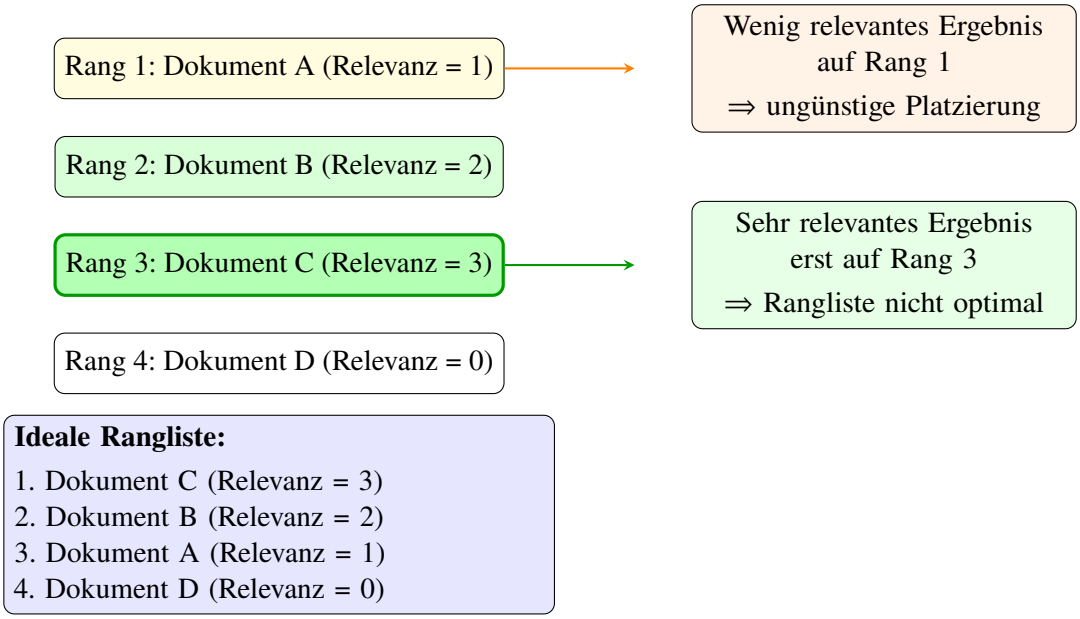


Abbildung 2.9: Beispiel für nDCG. Die Relevanz eines Ergebnisses wird unabhängig von seiner Rangposition betrachtet. Obwohl Dokument C die höchste Relevanz besitzt, erscheint es erst auf Rang 3. nDCG erkennt diese Fehlplatzierung und bewertet die Rangliste schlechter als eine ideal sortierte Rangliste.

Der Wertebereich liegt zwischen 0 und 1. Ein Wert von 1 bedeutet, dass die Ergebnisse optimal entsprechend der idealen Rangliste sortiert wurden.

2.4 Handlungsbedarf und Forschungsfragen

Die in Kapitel 1.2 beschriebene Herausforderung zeigt, dass die Suche nach geeigneten Makros innerhalb großer Makrobestände mit einem erheblichen zeitlichen und wirtschaftlichen Aufwand verbunden sein kann.

Gleichzeitig wurde in Kapitel 2 gezeigt, dass bereits verschiedene Verfahren zur Verarbeitung, Bewertung und Empfehlung von Informationen existieren.

Hieraus ergibt sich ein Handlungsbedarf zur Entwicklung eines Systems, das die vorhandenen Artikel- und Makrodaten automatisiert auswerten kann. Ein solches System soll technische Informationen aus Artikeln extrahieren, diese mit bereits bekannten Artikel-Makro-Zuordnungen vergleichen und anschließend geeignete Makros in Form einer Rangliste empfehlen.

Für die Entwicklung eines solchen Empfehlungssystems müssen zunächst die relevanten technischen Informationen innerhalb der verfügbaren Artikeldaten identifiziert und hinsichtlich ihrer

Eignung zur Beschreibung von Makros untersucht werden. Darauf aufbauend stellt sich die Frage, welcher Modellansatz die vorhandenen Informationen bestmöglich zur automatisierten Makroempfehlung nutzen kann. Anschließend muss bewertet werden, welche Empfehlungsqualität der ausgewählte Ansatz erreicht und inwieweit die empfohlenen Makros hinsichtlich ihrer Relevanz korrekt priorisiert werden können. Abschließend ist zu untersuchen, welchen praktischen Nutzen ein solches Empfehlungssystem für die Elektrokonstruktion besitzt und ob die in Kapitel 1.3 beschriebenen Such- und Auswahlaufwände tatsächlich reduziert werden können.

Vor diesem Hintergrund ergibt sich folgende übergeordnete Forschungsfrage:

> **RQ0:** Inwieweit eignet sich ein KI-basiertes Empfehlungssystem auf Basis vorhandener Artikel-Makro-Zuordnungen zur automatisierten Empfehlung geeigneter Makros für neue Elektrokonstruktionsaufgaben?

Zur Beantwortung dieser Forschungsfrage werden folgende Teilforschungsfragen untersucht:

RQ1: Welche technischen Informationen aus Artikeln eignen sich zur Unterscheidung von Makros und können als Trainings- und Evaluationsbasis für ein Empfehlungssystems verwendet werden?

RQ2: Welcher Modellansatz eignet sich zur Abbildung und Nutzung vorhandener Artikel-Makro-Zuordnungen für die automatisierte Empfehlung geeigneter Makros?

RQ3: Welche Empfehlungsqualität erreicht der ausgewählte Modellansatz hinsichtlich Trefferwahrscheinlichkeit und Rankingqualität?

RQ4: Welchen Einfluss besitzt das entwickelte Empfehlungssystem auf den Suchaufwand und die Wiederverwendung bestehender Makros in der Elektrokonstruktion?

Das Ziel dieser Arbeit besteht folglich darin, die Machbarkeit eines Empfehlungssystems zu untersuchen, einen geeigneten Modellansatz auf Basis der verfügbaren Daten auszuwählen, dessen Empfehlungsqualität zu evaluieren und den potenziellen Nutzen für die Elektrokonstruktion zu bewerten.

3 Vorgehensweise

3.1 Methodisches Vorgehen

Zur Beantwortung der in Kapitel 2.4 formulierten Forschungsfragen wird die Design Science Research Methodology (DSRM) nach Peffers et al. verwendet (Peffers u. a. 2007). Die DSRM beschreibt einen strukturierten Forschungsprozess zur Entwicklung und Evaluation technischer Artefakte und eignet sich insbesondere für die Entwicklung von Informationssystemen. Das Ziel der Methodik besteht darin, eine Problemstellung systematisch zu analysieren, daraus Anforderungen abzuleiten, ein geeignetes Artefakt zu entwickeln und dessen Nutzen durch eine Evaluation nachzuweisen. Der von Peffers et al. beschriebene Forschungsprozess umfasst die folgenden Schritte:

1. Problemidentifikation und Motivation
2. Definition der Lösungsziele
3. Design und Entwicklung des Artefakts
4. Demonstration der Lösung
5. Evaluation des Artefakts
6. Kommunikation der Ergebnisse

3.2 Iterativer Entwicklungsprozess

Innerhalb der Design-, Entwicklungs- und Evaluationsphase wird die DSRM durch die Prinzipien des Action Design Research (ADR) nach Sein et al. ergänzt (Sein u. a. 2011). ADR verbindet die Entwicklung eines Artefakts mit dessen fortlaufender Analyse, Validierung und Verbesserung. Während die DSRM den übergeordneten Forschungsrahmen bereitstellt, definiert ADR einen iterativen Entwicklungszyklus zur schrittweisen Verbesserung des Empfehlungssystems. Der iterative Entwicklungsprozess umfasst die folgenden Schritte:

1. Analyse des aktuellen Systemzustands
2. Recherche fachlicher und technischer Lösungsansätze
3. Konzeption neuer Komponenten oder Optimierungen
4. Implementierung der entwickelten Ansätze

5. Durchführung von Tests und Validierungen
6. Evaluation anhand definierter Qualitätsmetriken
7. Interpretation der Ergebnisse
8. Ableitung weiterer Verbesserungen

Die Ergebnisse einer Iteration bilden jeweils die Grundlage für die nachfolgende Iteration. Dadurch entsteht ein kontinuierlicher Verbesserungsprozess, in dem Forschung, Entwicklung und Evaluation eng miteinander verzahnt sind. Die Kombination aus DSRM und ADR ermöglicht einerseits einen strukturierten wissenschaftlichen Forschungsprozess und andererseits eine iterative Optimierung des entwickelten Empfehlungssystems. Abbildung 3.1 veranschaulicht die Verzahnung beider Methodiken.

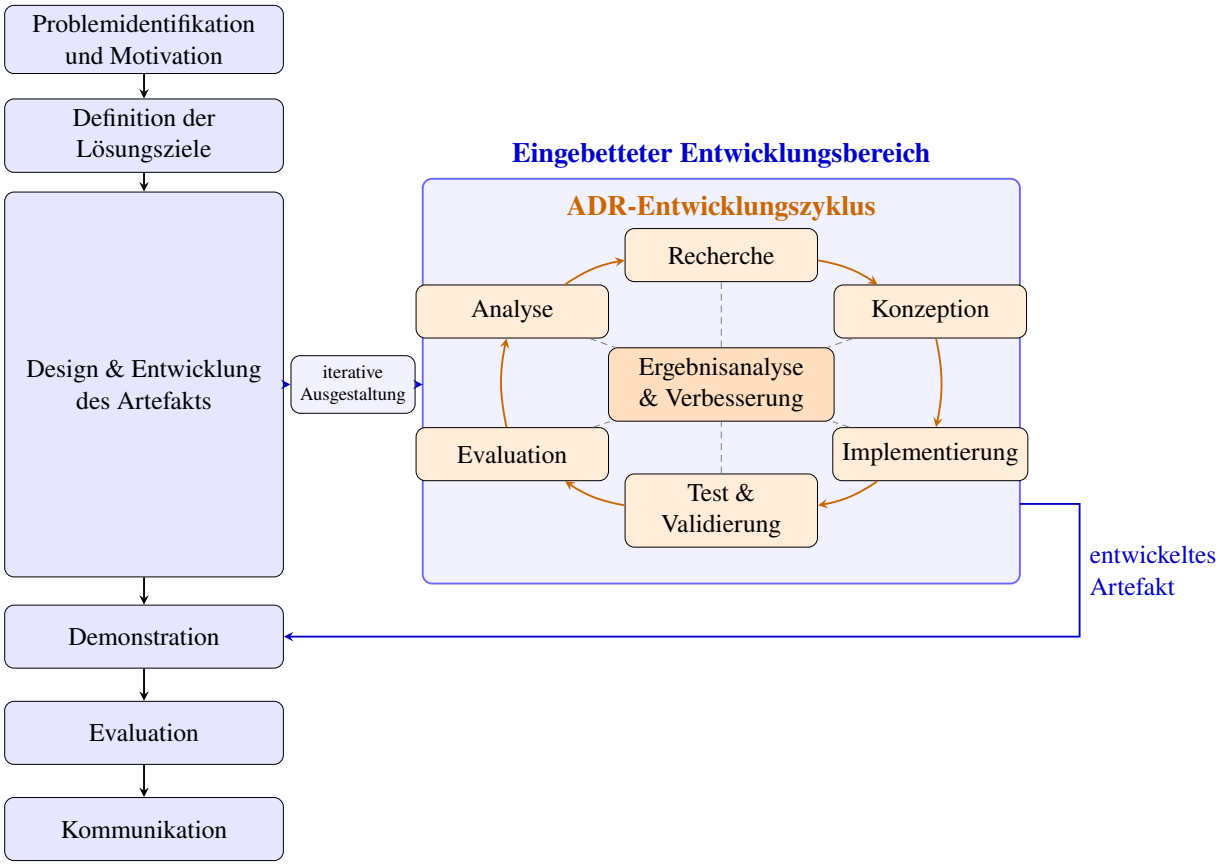


Abbildung 3.1: Verzahnung von DSRM und ADR. Die DSRM bildet den übergeordneten Forschungsrahmen, während ADR als iterativer Entwicklungszyklus innerhalb der Phase „Design & Entwicklung des Artefakts“ eingesetzt wird.

Die in Abbildung 3.1 dargestellte Verzahnung von DSRM und ADR bildet den methodischen Rahmen der vorliegenden Arbeit. Die Phasen „Problemidentifikation und Motivation“ sowie „Definition der Lösungsziele“ wurden bereits in Kapitel 1 und Kapitel 2 durchgeführt. Hierbei wurden die Herausforderungen der Makrosuche analysiert, die Vision des Empfehlungssystems entwickelt, der aktuelle Stand der Technik betrachtet und darauf aufbauend die Forschungsfragen RQ0 bis

RQ4 abgeleitet, welche als Zielhierarchie für die vorliegende Arbeit dienen. Die nachfolgenden Kapitel bilden die Design-, Entwicklungs-, Demonstrations- und Evaluationsphase der DSRM ab. Dabei entsteht in jedem Kapitel ein Artefakt, das als Grundlage für die jeweils nachfolgenden Arbeitsschritte dient. Die Zuordnung von Forschungsfragen, DSRM-Phasen, Artefakten und Kapiteln ist in Tabelle 3.1 dargestellt.

Tabelle 3.1: Zuordnung von Forschungsfragen, DSRM-Phasen, Artefakten und Kapiteln

Kapitel	DSRM-Phase	Forschungsfrage	Artefakt
4	Design & Entwicklung	RQ1	A1: Trainings- und Evaluationsbasis
5	Design & Entwicklung	RQ2	A2: Ausgewählter Modellansatz
6	Design & Entwicklung	RQ2	A3: Systemarchitektur
7	Design & Entwicklung	RQ2	A4: Trainiertes Empfehlungssystem
8	Demonstration & Evaluation	RQ3	A5: Evaluationsergebnisse
9	Evaluation	RQ4	A6: Nutzen- und Wirtschaftlichkeitsbewertung
10	Kommunikation	RQ0	A7: Beantwortung der Forschungsfragen

Die Bearbeitung der Arbeit erfolgt entlang dieser Artefaktkette.

Kapitel 4 untersucht zunächst die verfügbaren Artikel-, Makro- und Zuordnungsdaten und dient der Beantwortung von Forschungsfrage RQ1. Das Ergebnis dieses Kapitels ist die Trainings- und Evaluationsbasis des Empfehlungssystems (Artefakt A1).

Anschließend werden in Kapitel 5 verschiedene KI-basierte Modellansätze analysiert und gegenübergestellt. Hierbei werden insbesondere die Large Language Models mit Fine-Tuning, Sentence Transformer, Embedding-Modelle mit Triplet Loss sowie Learning-to-Rank-Verfahren betrachtet (Vergleiche Kapitel **??**). Ziel dieses Kapitels ist die Auswahl eines geeigneten Modellansatzes zur Beantwortung von Forschungsfrage RQ2. Das Ergebnis bildet Artefakt A2.

Auf Grundlage des ausgewählten Modellansatzes wird in Kapitel 6 die Architektur des Empfehlungssystems entwickelt. Dieses Architekturkonzept bildet Artefakt A3 und spezifiziert die Komponenten, Datenflüsse und Schnittstellen des Systems.

Kapitel 7 beschreibt die Implementierung und das Training des ausgewählten Modells. Hierzu gehören die Erstellung der Trainingsdaten, die Modellkonfiguration, die Hyperparameteroptimierung sowie der eigentliche Trainingsprozess. Das Ergebnis dieses Kapitels ist das trainierte Empfehlungssystem (Artefakt A4).

Die Demonstration und Evaluation des entwickelten Artefakts erfolgt in Kapitel 8. Hierzu werden die im Kapitel 2.3 vorgestellten Bewertungsmetriken Recall@k, Mean Reciprocal Rank und nDCG verwendet. Das Ergebnis dieser Untersuchungen bildet Artefakt A5 und dient der Beantwortung von Forschungsfrage RQ3.

Kapitel 9 bewertet die erzielten Ergebnisse hinsichtlich ihres praktischen Nutzens für die Elektrokonstruktion. Hierzu werden insbesondere Suchaufwand, Wiederverwendung von Makros und wirtschaftliche Auswirkungen (Vergleiche Kapitel 1.3) betrachtet. Dieses Kapitel liefert die notwendigen Erkenntnisse zur Beantwortung von Forschungsfrage RQ4 und erzeugt Artefakt A6.

Abschließend werden in Kapitel 10 die Ergebnisse der Arbeit zusammengeführt. Die Forschungsfragen RQ0 bis RQ4 werden beantwortet, die Grenzen des entwickelten Systems diskutiert und mögliche zukünftige Erweiterungen aufgezeigt. Das Ergebnis dieses Kapitels bildet Artefakt A7.

Die innerhalb der DSRM-Phasen „Design & Entwicklung“ sowie „Evaluation“ entstehenden Artefakte werden nicht linear entwickelt, sondern durchlaufen die in Abbildung 3.1 dargestellten ADR-Zyklen. Insbesondere während der Datenanalyse, Modellauswahl, Implementierung und Evaluation erfolgen wiederholte Iterationen aus Analyse, Recherche, Konzeption, Implementierung und Bewertung. Die Ergebnisse jeder Iteration fließen dabei unmittelbar in die Weiterentwicklung der nachfolgenden Artefakte ein. Dadurch entsteht ein iterativer Entwicklungsprozess, in dem die Forschungsfragen schrittweise beantwortet und die einzelnen Artefakte kontinuierlich verbessert werden.

4 Datenanalyse und Trainingsvorbereitung

Die in Kapitel 2.4 formulierte Forschungsfrage RQ1 untersucht, welche technischen Informationen aus Artikeln geeignet sind, um Makros voneinander zu unterscheiden und für die Trainingsbasis eines Empfehlungssystems geeignet sind.

Zur Beantwortung dieser Forschungsfrage werden in diesem Kapitel zunächst die verfügbaren Datenquellen analysiert. Anschließend werden die Schritte der Datenaufbereitung beschrieben, die notwendig sind, um eine für das Empfehlungssystem geeignete Basis an Daten zu erzeugen.

4.1 Datenbasis

4.1.1 Artikeldaten

Die Artikeldaten liegen in XML-Dateien vor. XML (*Extensible Markup Language*) ist ein standardisiertes Format zur hierarchischen Speicherung von Informationen. Durch die hierarchische Struktur können sowohl einfache Stammdaten wie Herstellerinformationen als auch komplexe Informationen wie Spannungen, Ströme oder Leistungen abgebildet werden. Die Analyse der Artikeldaten hat ergeben, dass sie eine Vielzahl von technischen Informationen enthalten, darunter:

- Herstellerinformationen,
- Typ- und Bestellnummern,
- Spannungen,
- Ströme,
- Leistungen,
- Kommunikationsschnittstellen,
- technische Hinweise,
- Referenzen auf zugehörige Makros.

Um diese technischen Informationen anhand des Motorbeispiels aus Kapitel 1.1 zu veranschaulichen, soll hierfür ein Motor von SEW betrachtet werden. Abbildung 4.1 zeigt die beispielhafte Struktur eines XML-Artikels für diesen Motor.

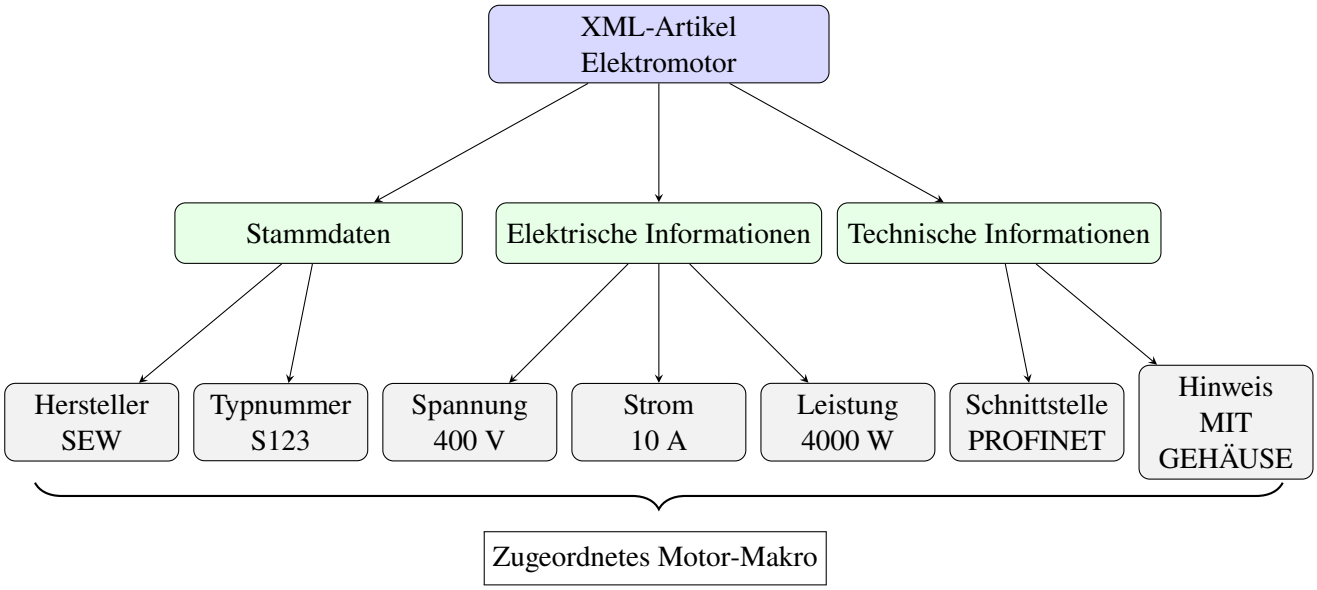


Abbildung 4.1: Beispielhafte Struktur eines XML-Artikels für einen Elektromotor. Die hierarchische Organisation ermöglicht die Speicherung von Herstellerinformationen, technischen Informationen sowie Referenzen auf zugehörige Makros.

4.1.2 Makrodaten

Neben den Artikeln werden die zugehörigen Makros analysiert. Die Makros sind im .ema-Format gespeichert. Dieses Format entspricht einem XML-basierten Schema, das sowohl grafische als auch technische Informationen strukturiert abbildet.

Hierzu zählen:

- Anschlussstrukturen,
- Makrovarianten,
- Bezeichnungen.

Während die Artikeldaten beschreiben, welche technischen Informationen ein Gerät besitzt, beschreiben die Makros die technische Umsetzung dieser Informationen innerhalb eines Schaltplans. Beide Informationsquellen ergänzen sich daher gegenseitig.

Die Bezeichnungen eines Makros werden nach folgendem Muster aufgebaut: Makro-Familie-Variante (z. B. M1_F1_V1) Dabei gibt Makro die Nummerierung des Makros selbst an, die Familie beschreibt eine Unterkategorie des Makros und die Variante kennzeichnet unterschiedliche Ausführungen innerhalb dieser Familie. Im Fall des Motorbeispiels aus Kapitel 1.1 lässt sich die Bezeichnung M1_F1_V1 wie folgt interpretieren:

- M1: Motor

- F1: Gleichstrommotor
- V1: Mit Kühlung

Die Abbildung 4.2 gibt die Struktur eines Motor-Makros im .ema-Format wieder.

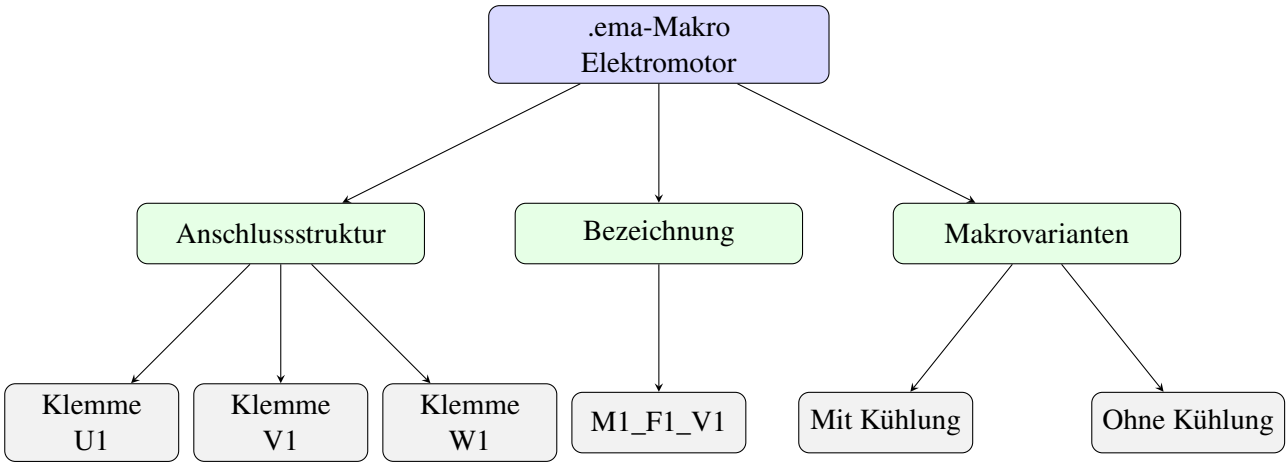


Abbildung 4.2: Beispielhafte Struktur eines Motor-Makros im .ema-Format. Das Makro beschreibt die technische Umsetzung eines Elektromotors im Schaltplan und enthält Informationen zu Anschlussstrukturen, Bezeichnungen und Makrovarianten.

4.1.3 Artikel-Makro-Zuordnungen

Die Zuordnung von Artikel zu Makro erfolgt durch die Referenzierung des Makros über die Artikel-Eigenschaft P_ARTICLE_MACRO. Diese Zuordnung kann als n zu 1 Beziehung betrachtet werden, da mehrere Artikel auf dasselbe Makro verweisen können.

Bildlich kann dies durch folgende Abbildung 4.3 dargestellt werden:

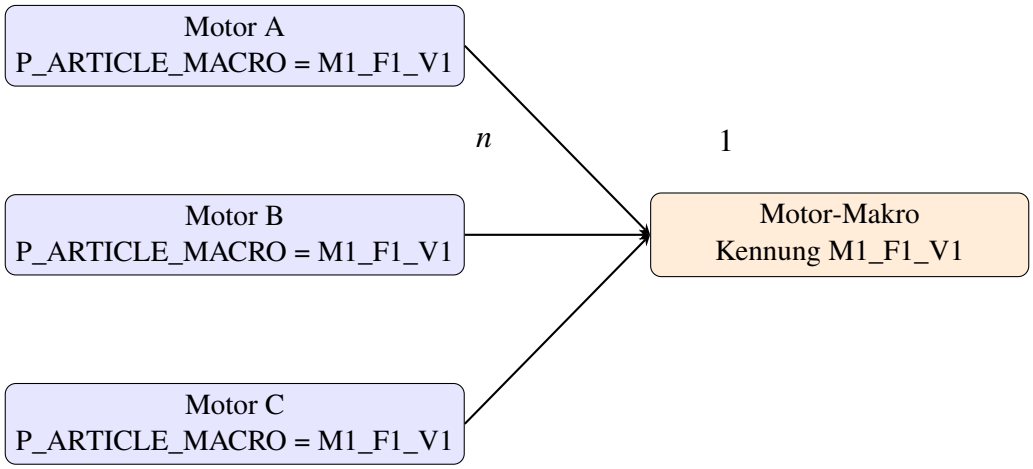


Abbildung 4.3: Beispiel einer n:1-Beziehung zwischen Artikeln und Makros. Mehrere Artikel verweisen über die Eigenschaft P_ARTICLE_MACRO auf dasselbe Makro.

4.2 Datenaufbereitung

Die vorliegende Struktur der Daten im XML-Format hat den Vorteil, dass Informationen nicht manuell ausgelesen und weiterverarbeitet werden müssen. Dadurch kann die Vorbereitung der Daten automatisiert erfolgen, so dass eine nachvollziehbare und reproduzierbare Datenaufbereitung gewährleistet werden kann. Zudem unterstützt diese Art der Datenaufbereitung die Vision aus Kapitel 1.4, ein automatisiertes Empfehlungssystem zu entwickeln.

Die in Kapitel 2.2 beschriebenen Modellansätze lernen statistische Zusammenhänge zwischen den Eingabedaten (Artikel) und den Ausgabedaten (Makros). Enthalten diese Daten fehlerhafte oder wiedersprüchlichen Informationen, so werden diese mit erlernt. Das Ziel der Datenaufbereitung besteht daher darin, die technische Qualität der Datenbasis zu erhöhen und gleichzeitig Störeinflüsse zu reduzieren

4.2.1 Entfernung unvollständiger Artikel- und Makrodaten

In diesem Schritt werden alle Artikel und Makros entfernt, die unvollständige Informationen enthalten. Dies betrifft fehlende technische Merkmale von Artikeln wie Spannung, Leistung oder Strom sowie fehlende Artikel-Makro-Zuordnungen und fehlerhafte Makroberschreibungen.

Abbildung 4.4 zeigt die Unterscheidung zwischen vollständigen und unvollständigen Artikeldaten:

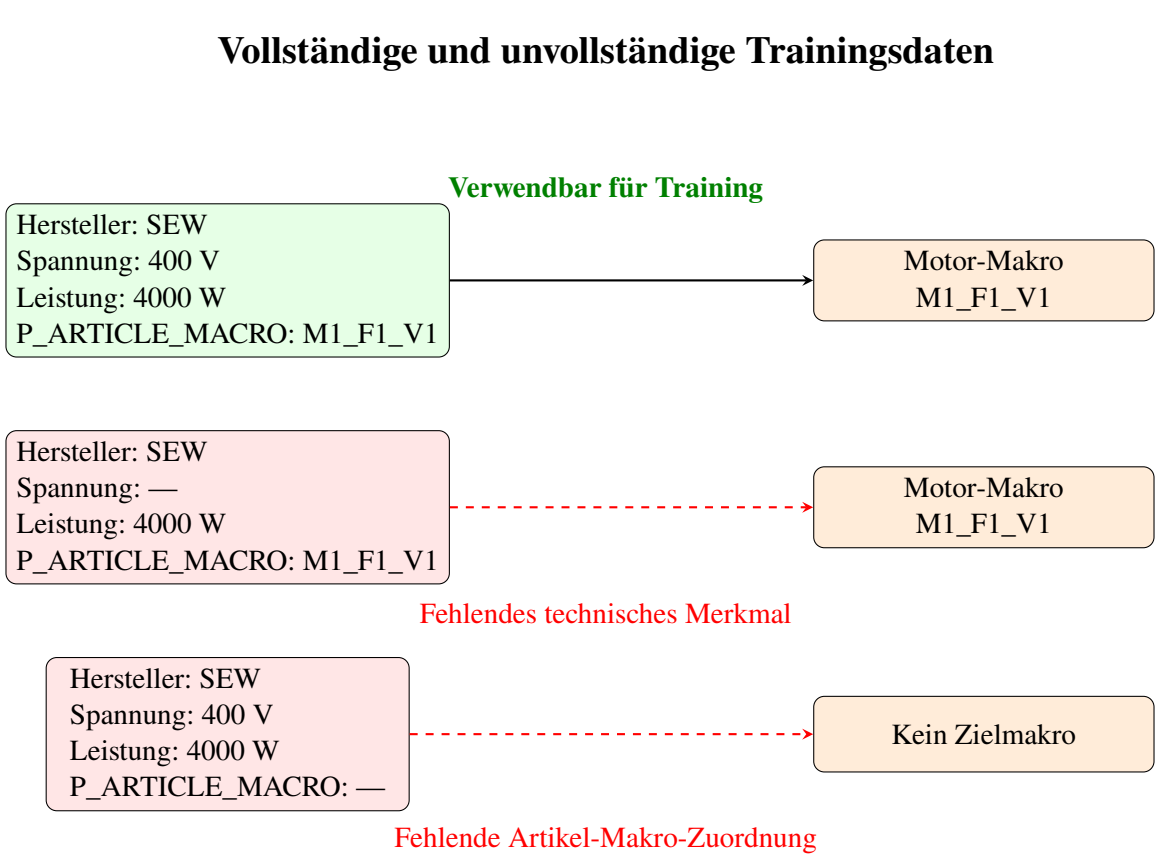


Abbildung 4.4: Beispiel für vollständige und unvollständige Artikeldaten. Fehlende technische Merkmale oder fehlende Artikel-Makro-Zuordnungen verhindern eine zuverlässige Verwendung der Datensätze im Training und führen daher zu deren Entfernung.

4.2.2 Entfernung redundanter Artikel- und Makrodaten

Hierbei werden Artikel und Makros identifiziert, die inhaltlich identisch sind. Identische Inhalte führen zu einer Verzerrung der Datenbasis, da hierdurch die Gewichtung bestimmter Artikel erhöt wird. In einem späteren Lernprozess wird hierdurch das Wissen eines Modells verzerrt. Das Modell erlernt dadurch ein Vorurteil auf übergewichteten Artikel, welche bei neuen Anfragen bevorzugt vorhergesagt werden.

Abbildung 4.5 zeigt die Auswirkung der redundanter Artikel- und Makrodaten auf deren Gewichtung:

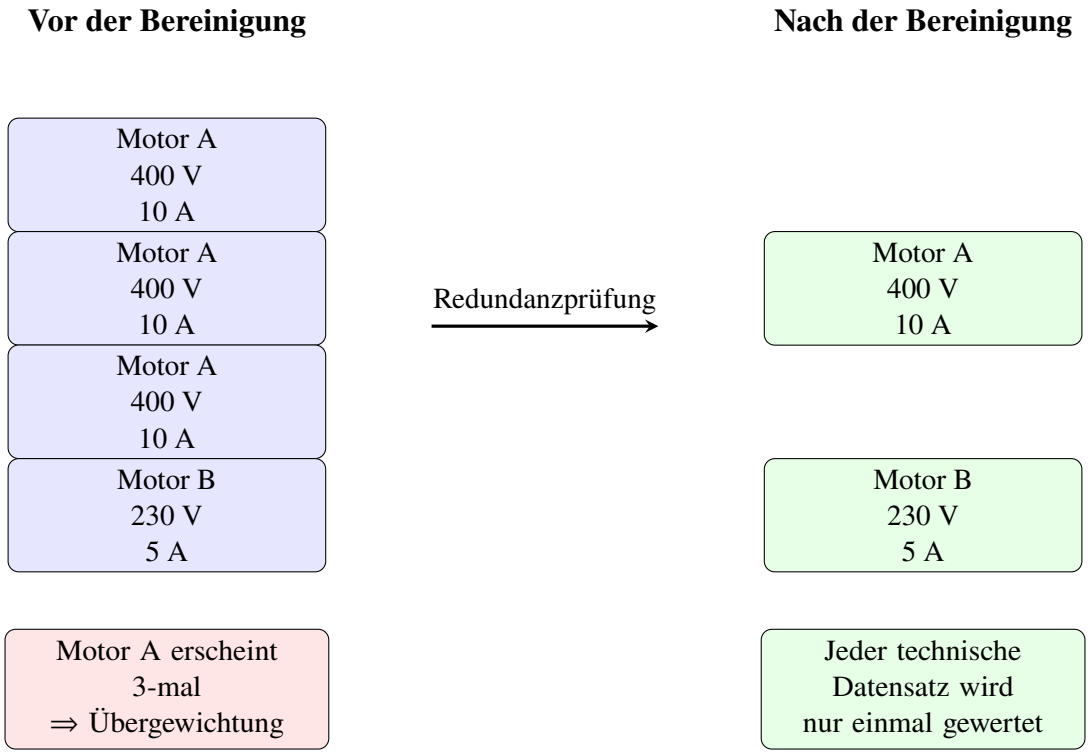


Abbildung 4.5: Auswirkung redundanter Artikel- und Makrodaten. Mehrfach vorhandene Datensätze erhöhen künstlich die Gewichtung bestimmter technischer Informationen. Durch die Entfernung redundanter Daten wird diese Verzerrung reduziert.

4.2.3 Entfernen nicht nutzbarer Metadaten

Metadaten sind zusätzliche Informationen über Daten, die nicht direkt mit dem Inhalt dieser Daten zusammenhängen. Beispiele hierfür sind Erstellungsdatum, Änderungsdatum oder Autor. Diese Metadaten sind für die Erkennung technischer unterschiede zwischen Artikeln und Makros nicht relevant und führen zu sogenanntem Rauschen in den Daten. Dieses Rauschen kann als Informationsüberflutung betrachtet werden, was dazu führt, dass ein Modell später Wissen über irrelevante Informationen erlernt und dadurch die Erkennung relevanter technischer Unterschiede erschwert wird.

Anhand der Abbildung 4.6 soll verdeutlicht werden, wie sich Metadaten von relevanten technischen Informationen unterscheiden:

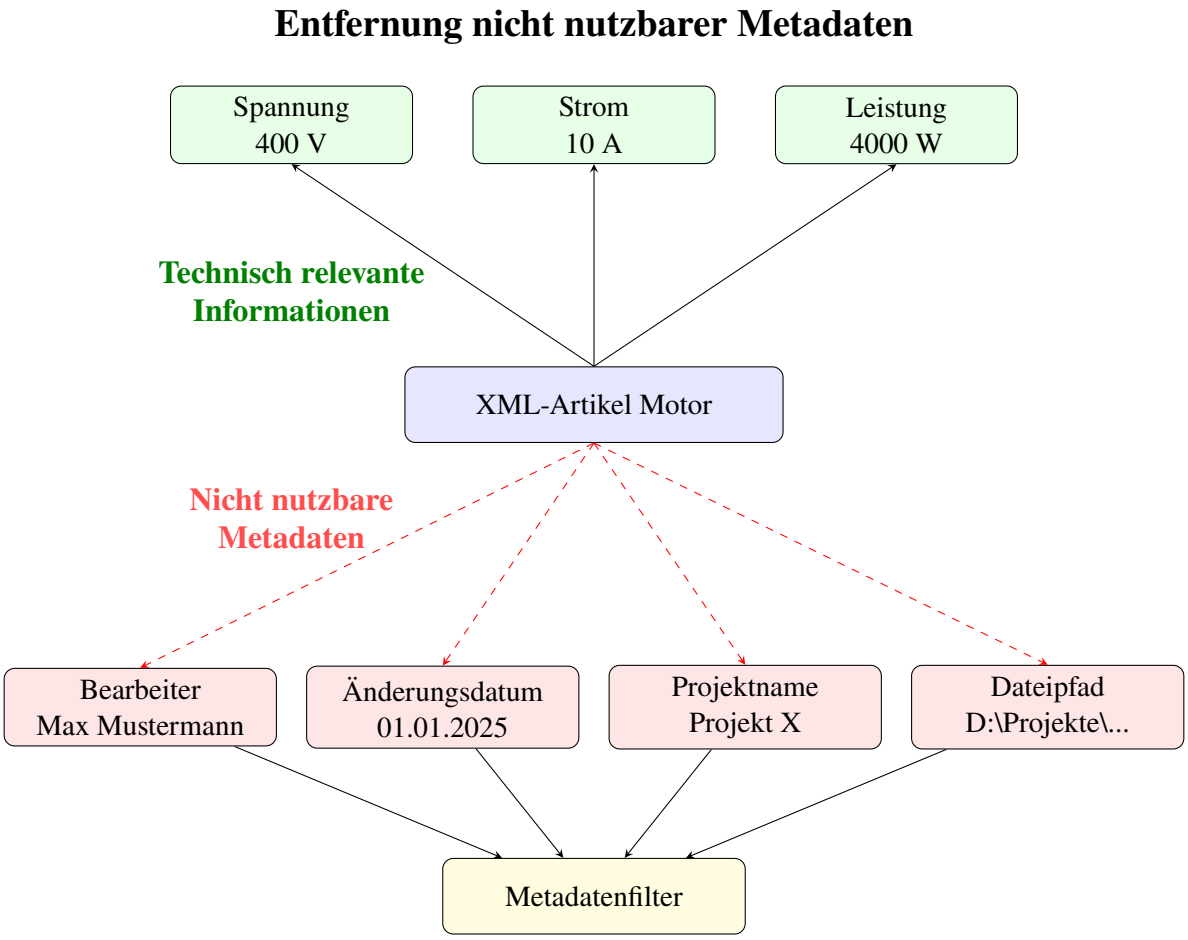


Abbildung 4.6: Entfernung nicht nutzbarer Metadaten. Technisch relevante Merkmale bleiben erhalten, während Metadaten ohne technischen Informationsgehalt entfernt werden.

4.2.4 Normaliserung von Einheiten

Die Technischen Informationen der Artikel liegen in unterschiedlichen Einheiten oder Schreibweisen vor. Ohne zusätzliches Wissen was die Einheiten bedeuten, ist kein direkter Vergleich dieser Werte möglich. Diese fehlende Standardisierung führt zu einer Streuung identischer technischer Informationen, welche ein Rauschen in den Artikeldaten erzeugt. Deswegen werden die technischen Informationen aus den Artikeln auf gemeinsame Basiseinheiten normiert.

Folgende Abbildung 4.7 soll dazu diesen Sachverhalt zu veranschaulichen:

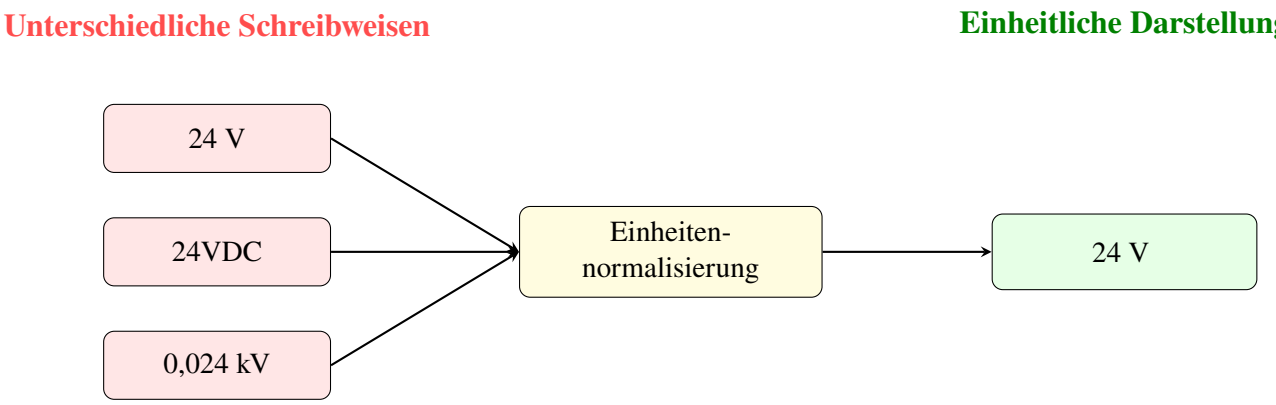


Abbildung 4.7: Beispiel für die Normalisierung technischer Einheiten. Unterschiedliche Schreibweisen derselben Spannung werden in eine einheitliche Darstellung überführt, wodurch die Vergleichbarkeit der Daten verbessert wird.

4.2.5 Erstellung von Klassifikationen

Die Vorhandenen technischen Informationen der Artikeldaten liegen in unterschiedlichen Wertebereichen vor. Dieses Wissen über die Wertebereiche ermöglicht es, die technischen Informationen zu Klassifikationen zusammenzufassen. Eine Klassifikation fasst Werte in bestimmte Kategorien zusammen, wodurch die Vergleichbarkeit und die Analyse der technischen Informationen erleichtert wird. Diese Klassifikationen stellen ein zusätzliches Wissen in den technischen Informationen eines Artikels dar, welches für die Unterscheidung zwischen mehreren Artikeln nützlich ist. Gleichzeitig stellt eine Klassifkikation sicher, dass ähnliche Informationen in dieselbe Kategorie fallen, wodurch eine Vorsortierung der Artikeldaten erfolgt und somit der Lernprozess des Modells Zielgerichteter gestaltet werden kann.

Die folgende Abbildung 4.8 zeigt, wie die Klassifikation von Spannungen dargestellt wird:

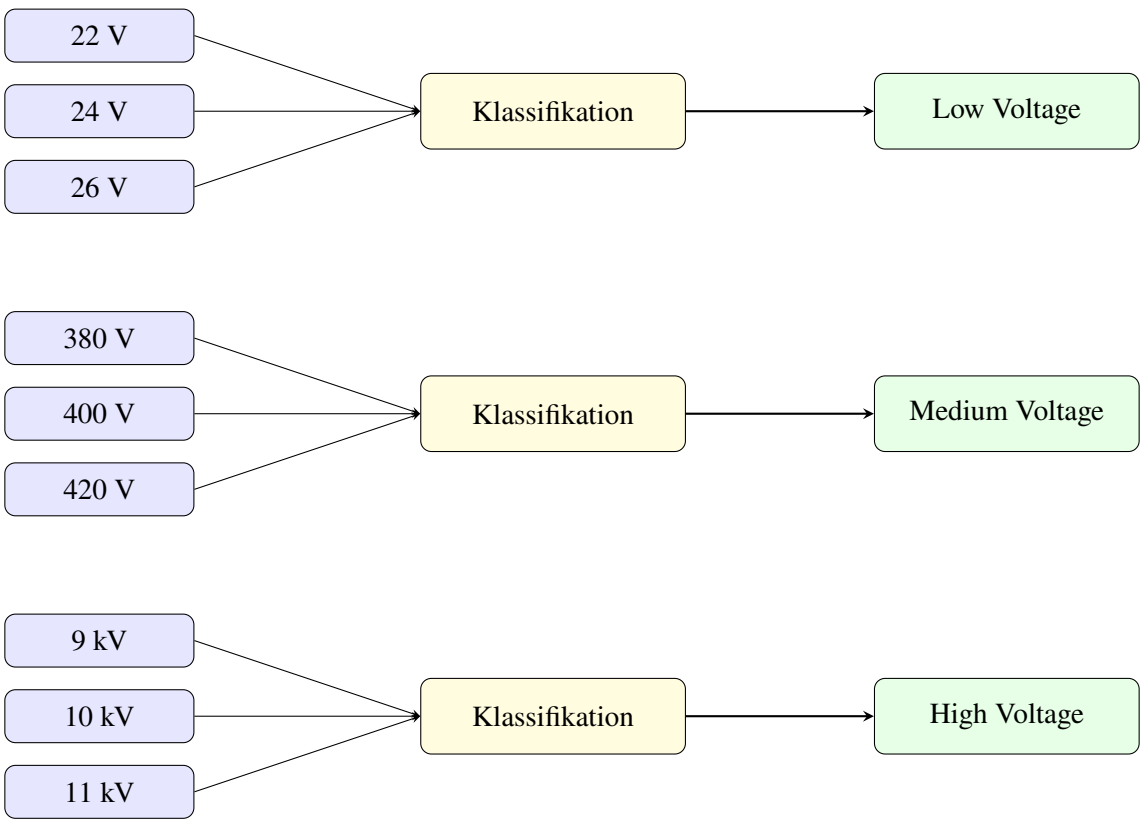


Abbildung 4.8: Beispiel für die Klassifikation technischer Spannungswerte. Mehrere technisch ähnliche Spannungswerte werden durch eine Klassifizierung zu gemeinsamen Spannungsklassen zugeordnet. Dadurch können ähnliche Geräte trotz leicht unterschiedlicher Spannungswerte gemeinsam betrachtet werden.

4.2.6 Ergebnis der Datenaufbereitung

Die Aufteilung der Artikel- und Makrodaten erfolgt unter Berücksichtigung der vorangegangenen Datenaufbereitung (siehe Abschnitt 4.2). Nach der Anwendung der in der Datenaufbereitung durgeführten Bereinigung, Normalisierung und Klassifizierung hat sich der Ursprüngliche Bestand an Artikeln und Makros von:

- .... Artikel,
- .... Makros,

auf

- 1.559 Artikel,
- 375 Makros,

reduziert.

4.3 Überführung in ein Trainingsformat

Modelle wie sie in Kapitel 2.2 beschrieben werden, können technische Informationen nicht direkt aus XML-Dateien verarbeiten. Um diese Informationen nutzbar zu machen, müssen sie in Embeddings (vergleiche Kapitel 2.1.3) überführt werden. Deswegen werden die technischen Informationen aus Artikel- und Makrodaten in numerische Vektoren überführt, die als Eingabe für ein Modell dienen können. Die vorliegenden technischen Informationen werden in drei separate Vektoren überführt: den Artikelvektor, den Makrovektor und den Lexikalvektor. Diese Trennung sorgt für eine weitere Klassifikation der technischen Informationen. Dadurch entsteht eine Vorstrukturierung, welche für eine Abgrenzung der einzelnen technischen Informationen sorgt.

4.3.1 Artikelvektor

Der Artikelvektor listet die technischen Informationen eines einzelnen Artikels auf. Folgende Merkmale konnten dabei konsistent aus dem vorhandenen Artikeldaten unter Beachtung der vorherigen Datenaufbereitung (Vergleiche Kapitel 4.2) extrahiert werden:

- Hersteller,
- Produktfamilie,
- Produkttyp,
- Spannungen,
- Ströme,
- Leistungen,
- Frequenzen,
- Kommunikationsstandards,
- IP-Schutzklassen,
- Kanalzahlen,
- Anschlussprofile.

Die Abbildung 4.9 zeigt die Struktur und den Aufbau des Artikelvektors anhand des Motorbeispiels:

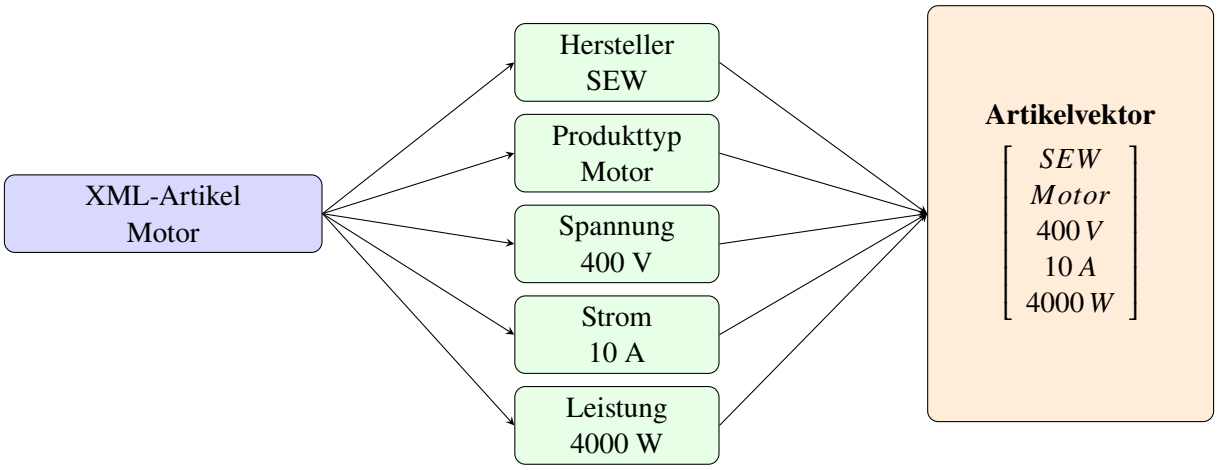


Abbildung 4.9: Erzeugung eines Artikelvektors aus technischen Informationen eines Motorartikels. Die extrahierten Merkmale werden zu einer gemeinsamen technischen Repräsentation zusammengeführt und dienen als Eingabe für das Empfehlungssystem.

4.3.2 Makrovektor

Der Makrovektor beschreibt die Struktur und den Aufbau eines Makros. Aus den vorhandenen Makrodaten werden analog zum Artikelvektor folgende Merkmale extrahiert:

- Funktionskategorien,
- Funktionsgruppen,
- Anschlussrollen,
- IO-Schemata,
- Kanalstrukturen,
- Signalanzahlen,
- Varianten,
- Symbolbibliotheken,
- Makrokategorien.

Im Bezug auf das Motorbeispiel zeigt folgende Abbildung 4.10 den Aufbau des Makrovektor:

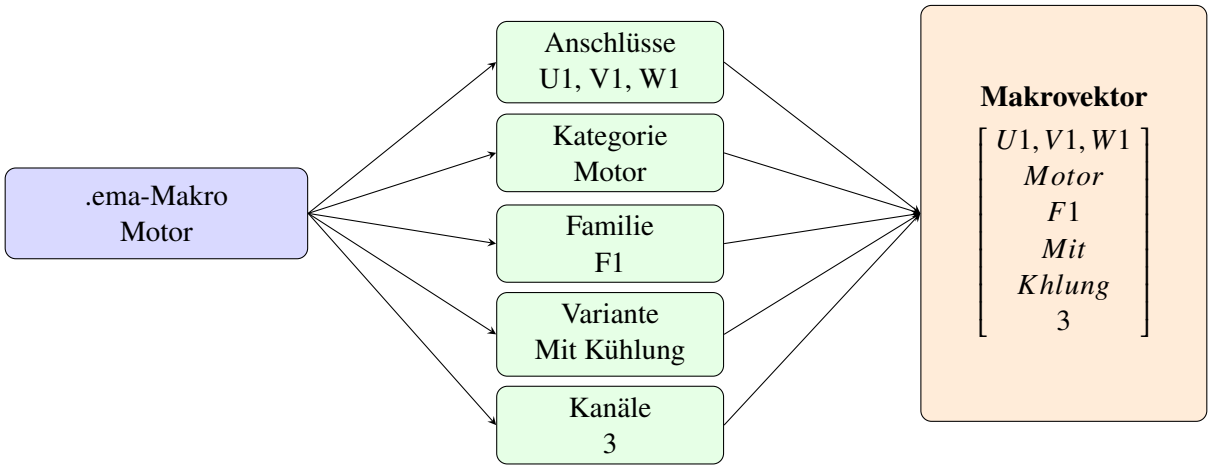


Abbildung 4.10: Erstellung des Makrovektor. Aus dem .ema-Makro werden strukturelle Informationen extrahiert und in einen numerischen Vektor überführt, der die technische Struktur des Makros für das Empfehlungssystem beschreibt.

4.3.3 Lexikalvektor

Nicht alle technischen Informationen liegen in Vorstrukturierter Form in den vorhandenen Artikel- und Makrodaten vor. Technische Hinweise in einem Artikel oder Makro liegen in Textform vor. Diese textbasierten technischen Informationen werden getrennt in einem eigenen Vektor erfasst, um ein weiters Rauschen aufgrund der Vermischung von Informationen zu vermeiden.

Aus den vorhandenen Artikel- und Makrodaten lassen sich folgende textbasierten Merkmale extrahieren:

- Typbezeichnungen,
- Produktserien,
- Kommunikationskennungen,
- technischen Beschreibungen.

Abbildung 4.11 stellt die Erstellung des Lexikalvektors für das Beispiel des Motors dar:

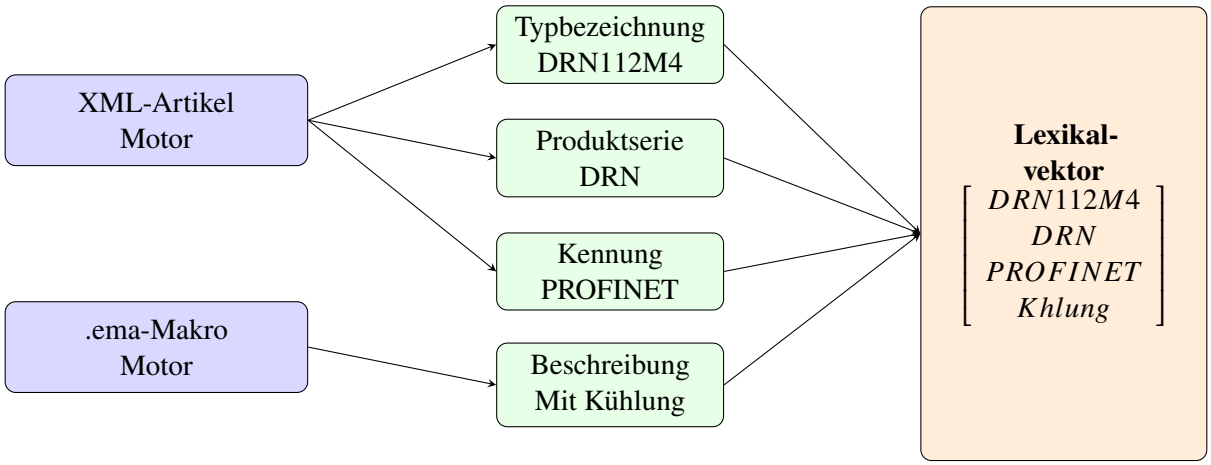


Abbildung 4.11: Erstellung des Lexikalvektors. Textuelle technische Informationen aus Artikel- und Makrodaten werden extrahiert und in einem separaten Vektor zusammengeführt. Der Lexikalvektor ergänzt die strukturierten Informationen des Artikel- und Makrovektors um technische Begriffe und Bezeichnungen.

4.4 Erstellung des Datensplits

4.4.1 Trainings-, Validierungs- und Testdaten

Ein Datensplit ist die Aufteilung vorhandener Daten in verschiedene Teilmengen, um damit den Lernprozess eines Modells überwachen, bewerten und überprüfen zu können. Hierfür werden die Artikel- und Makrodaten in Trainings-, Validierungs- und Testdaten aufgeteilt. Die Trainingsdaten dienen dem Lernprozess des Modells, wodurch die Zusammenhänge zwischen den technischen Informationen erlernt werden. Die Validierungsdaten werden verwendet, um die Parameter des Modells während des Lernprozesses zu überwachen und anzupassen. Die Testdaten dienen der abschließenden Bewertung des Modells, um dessen Leistungsfähigkeit auf unbekannte Artikeldaten zu überprüfen.

4.4.2 Konsistenzprüfung der Artikel- und Makrodaten im Datensplit

Um sicherzustellen, dass alle Artikel- und Makrodaten, welche in Validierungs- oder Testdaten vorkommen, bereits im Trainingsbestand vertreten sind, wird eine Konsistenzprüfung durchgeführt. Da für Artikel und Makro in einer n zu 1 Beziehung stehen (siehe Kapitel 4.1.3), können mehrere

Artikel zu einem Makro auf den gesamten Datensplit verteilt sein. Falls Artikel- oder Makrodaten in den Validierungs- oder Testdaten vorkommen, die nicht im Trainingsbestand vorhanden sind, werden diese umsortiert.

Die Umsortierung erfolgt dabei nach folgenden Regeln:

- Ein Makro muss immer durch einen Artikel in den Trainingsdaten vertreten sein.
- Ein Makro, das in den Validierungsdaten vorkommt, muss immer einen Artikel in den Trainingsdaten haben.
- Ein Makro, das in den Testdaten vorkommt, muss jeweils einen Artikel in den Trainings- und Validierungsdaten haben.
- Die restlichen Artikel von Makros werden entsprechend der Verteilung 60% Trainingsdaten, 20% Validierungsdaten und 20% Testdaten aufgeteilt.

Die konkrete Aufteilung von Trainings-, Validierungs- und Testdaten hängt von der Größe und Struktur des Datensatzes ab. Häufig werden Aufteilungen von Trainings-, Validierungs- und Testdaten wie 80/10/10, 70/20/10 oder 60/20/20 verwendet. Entscheidend ist dabei weniger die exakte prozentuale Verteilung als vielmehr die Vermeidung von Datenleckagen und die Sicherstellung einer aussagekräftigen Evaluation. Durch die oben genannten Regeln zur Konsistenzprüfung werden Artikel als erstens in die Trainingsdaten verschoben. Dadruch entseht eine Bevorzugung der Trainingsdaten gegenüber Validierungs- und Testdaten und somit das Risiko, dass zu wenig Testdaten vorhanden sind. Um diesem Risiko entgegenzuwirken, werden die restlichen Artikel nach dem 60/20/20-Verhältnis auf Trainings-, Validierungs- und Testdaten verteilt, um damit eine höhere Gewichtigkeit der Testdaten zu erreichen.

4.4.3 Ergebnis des Datensplits

Auf Grundlage des reduzierten Bestands aus Kapitel 4.2.6, sowie der Beachtung der Regeln zur Konsistenzprüfung aus Kapitel 4.4.2 ergibt sich der folgende Datensplit:

- 984 Trainingsartikel (63,1 %),
- 322 Validierungsartikel (20,7 %),
- 253 Testartikel (16,2 %).

Dieser Datensplit zeigt eine leichte Abweichung gegenüber der Zielkonfiguration von 70% Trainingsdaten, 20% Validierungsdaten und 10% Testdaten. Diese Abweichung ist für das Ziel dieser Arbeit akzeptabel, da sie eine aussagekräftigere Bewertung des Modells durch eine erhöhte Anzahl an Testdaten ermöglicht.

4.4.4 Architektur der Datenaufbereitung

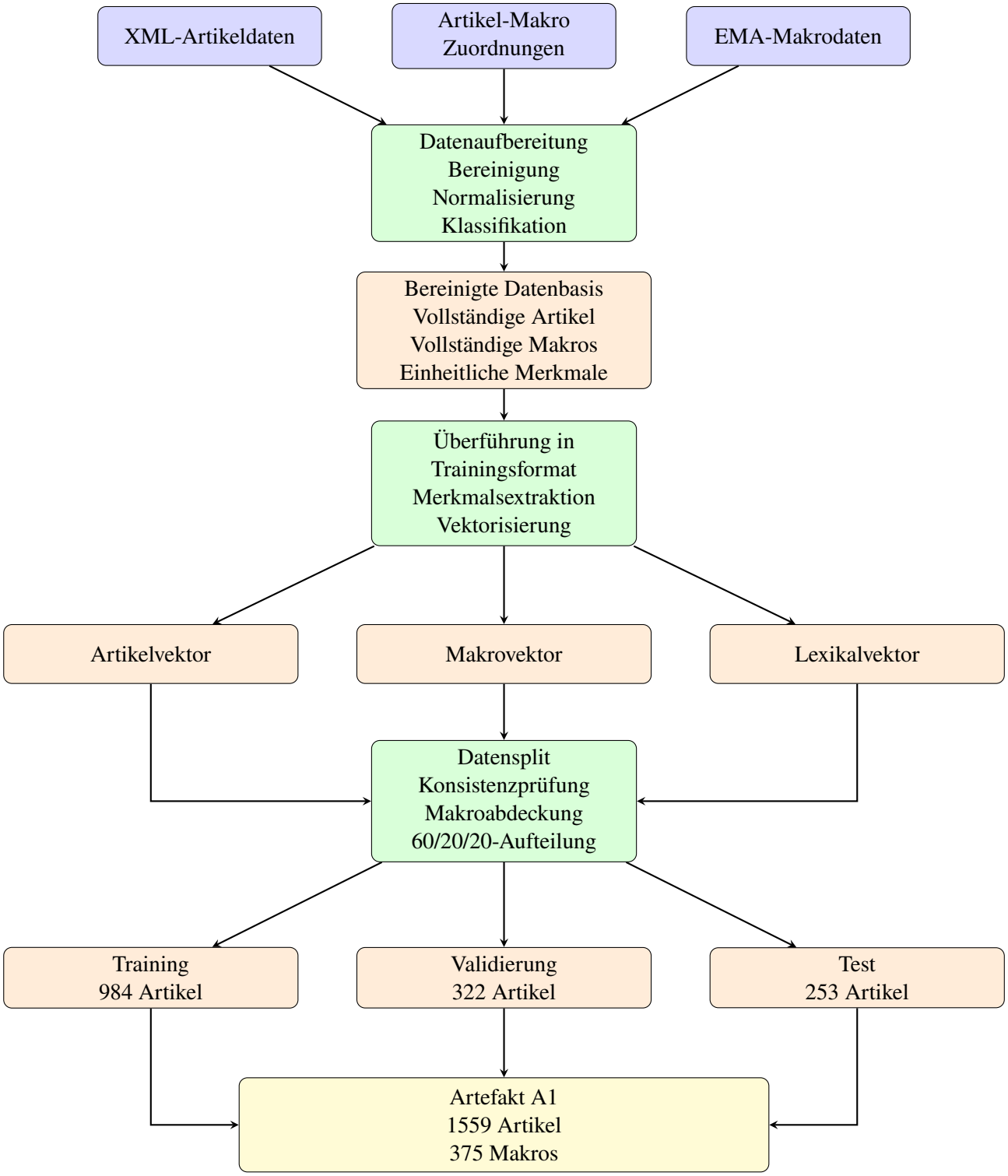


Abbildung 4.12: Architektur der Datenanalyse und Trainingsvorbereitung.

4.5 Kapitelergebnis

In diesem Kapitel wurden die verfügbaren Artikel- und Makrodaten sowie die vollständige Datenaufbereitung beschrieben. Die Analyse zeigt, dass die EPLAN-Artikelverwaltung eine umfangreiche technische Wissensbasis bereitstellt. Diese Informationen werden innerhalb der De extrahiert, normiert, qualitätsgesichert und in Artikel-, Makro- und Lexikalvektoren überführt. Zusätzlich wurden Mechanismen zur Leakage-Vermeidung und zur validen Aufteilung der Daten implementiert.

Artefakt A1 besteht damit aus:

- 1.559 qualitätsgesicherten Artikeldatensätzen,
- 375 Makrozielen,
- den vorhandenen Artikel-Makro-Zuordnungen,
- den erzeugten Artikelvektoren,
- den erzeugten Makrovektor,
- den erzeugten Lexikalvektoren,
- einem Merkmalsraum mit 2.381 Features,
- einem leakage-freien Trainings-, Validierungs- und Testsplit.

Damit liegt erstmals eine vollständige technische Repräsentation der vorhandenen Engineering-Wissensbasis vor. Dieses Artefakt beantwortet Forschungsfrage RQ1 und bildet die Grundlage für die Auswahl eines geeigneten Modellansatzes in Kapitel 5.

5 Auswahl des Modellansatzes

5.1 Bewertungsmatrix für Modellansätze

Kriterien:

Datenbedarf Trainingsaufwand Interpretierbarkeit Skalierbarkeit Eignung für Makroempfehlungen

5.2 Large Language Models

5.3 Sentence Transformer

5.4 Embedding-Modell mit Triplet Loss

5.5 Learning-to-Rank

5.6 Modellauswahl

5.7 Kapitelergebnis

Artefakt A2: Ausgewählter Modellansatz

6 Datenanalyse und Trainigsvorbereitung

6.1 Analyse der Ausgangsdaten

Eplan Dateien, .ema, Artikel, Makros, xml-format + Aufbau ...

6.2 Aufbereitung der Trainingsdaten

6.3 Datensplit

6.4 Trainingsbasis

7 Konzeption des Empfehlungssystems

7.1 Gesamtarchitektur

Aus Vision + Modellwahl

7.1.1 Datenfluss

Beispiel: Artikel ↓ Featureextraktion ↓ Kandidatenbildung ↓ Ranking ↓ Makroempfehlung

7.1.2 Komponentenmodell

Beschreibung der realen Pipeline 911 Data Preparation 913 Filtering 921 Split 931 Training 944 Analyse 949 Evaluation

7.1.3 Datenmodell

Artikelprofil Kandidatenprofil Anfrageprofil

7.2 Kapitelergebnis

Artefakt A3: Architektur des Empfehlungssystems

8 Implementierung des Modells

8.1 Kandidatenprofile

8.2 Feature Engineering

Artikelvektor Macro Shape Vector Lexical Vector

8.3 Trainingsmatrix

Nach der Erstellung der Kandidatenprofile werden für jede Anfrage die verfügbaren Makrokandidaten bewertet. Für jede Anfrage werden Anfrage-Kandidat-Paare erzeugt und anhand der Pairwise-Merkmale aus Kapitel **??** beschrieben. Die Referenzkonfiguration A08 verwendet insgesamt 375 Kandidatenprofile und 2.381 Merkmale. Für jede Anfrage werden alle 375 Kandidatenprofile vollständig bewertet.

Tabelle 8.1 fasst die Größenordnung der erzeugten Rankingmatrizen zusammen.

Tabelle 8.1: Dimensionen der erzeugten Rankingmatrizen

Split	Anfragen	Kandidaten je Anfrage	Matrixzeilen	Merkmale
Training	984	375	369.000	2.381
Validation	322	375	120.750	2.381
Test	253	375	94.875	2.381

Der erzeugte Artikel-Kandidat-Merkmalsraum ist sehr groß und viele Merkmalswerte besitzen den Wert null. Aus diesem Grund werden Sparse-Matrizen verwendet. Bei einer Sparse-Matrix werden nicht belegte Werte nicht vollständig gespeichert, wodurch der Speicherbedarf reduziert wird. Die numerischen Werte werden im 32-Bit-Gleitkommaformat $\mathrm{float32}$ verarbeitet.

8.4 Hyperparameteroptimierung

8.5 Hyperparameteroptimierung

Für die Hyperparameteroptimierung wird eine Grid-Search mit insgesamt 15 definierten Parameterkonfigurationen verwendet. Ein Grid-Search ist die systematische Erprobung von zuvor festgelegten Kombinationen an Modellparametern. Jede Kombination wird dabei trainiert und anhand des Validierungsbestands bewertet; anschließend wird anhand definierter Rankingmetriken die beste Konfiguration ausgewählt. Variiert werden dabei folgende Parameter:

- n_estimators,
- learning_rate,
- num_leaves,
- min_child_samples,
- lambdarank_truncation_level.

Die Tabelle 8.2 zeigt die untersuchten Hyperparameterkombinationen.

Tabelle 8.2: Variierte Hyperparameter der Grid-Search (Pipeline step 931 2026)

Parameter	**Wertebereich**
n_estimators	300 bis 900
learning_rate	0.025 bis 0.05
num_leaves	15 bis 127
min_child_samples	5 bis 80
lambdarank_truncation_level	5 bis 20

Dabei sind die Hyperparameter wie folgt zu verstehen:

- n_estimators**:** Anzahl der aufgebauten Entscheidungsbäume im Modell. Mehr Bäume = komplexere Zusammenhänge, aber mehr Rechenaufwand.
- learning_rate**:** Lernrate pro Trainingsschritt. Kleinere Werte führen zu vorsichtigerem, aber stabilerem Lernen.
- num_leaves**:** Maximale Anzahl der Blätter pro Baum. Mehr Blätter führen zu komplexeren Bäumen und erhöhen dadurch das Risiko von Overfitting.
- min_child_samples**:** Minimale Anzahl an Trainingsbeispielen in einem Blatt. Kleinere Werte = mehr Detailtreue pro Blatt, größere Werte = bessere Regularisierung (verhindert Overfitting).
- lambdarank_truncation_level**:** Begrenzung des Ranglistenbereichs, der durch LambdaRank optimiert werden soll (Top 10, Top 20).

8.6 Training

Das zentrale Element dieser Architektur basiert auf LightGBM. LightGBM ist eine Bibliothek für Gradient Boosting mithilfe von Entscheidungsbäumen. Dabei werden sequentiell Entscheidungsbäume erzeugt, welche die Fehler des bisherigen Modells schrittweise korrigieren. Für Rankingaufgaben ist der LGBMRanker besonders geeignet. Dieser ist auf Rankinggruppen sowie relative Ergebnisreihenfolgen spezialisiert und verwendet in diesem Projekt das LambdaRank-Verfahren (Microsoft Corporation 2026). Dieser basiert auf dem in Kapitel 2 beschriebenen LambdaRank-/LambdaMART-Prinzip.

Das Modell wird mit den Parametern

$$\text{objective} = \text{lambdarank} \qquad \text{und} \qquad \text{metric} = \text{ndcg} \tag{8.1}$$

trainiert (Pipeline step 931 2026).

Der Parameter objective legt das Optimierungsziel des Modells fest. Mit lambdarank wird die Optimierung der Kandidatenreihenfolge durch LambdaRank aktiviert. Der Parameter metric bestimmt die verwendete Rankingmetrik der Bewertung; ndcg steht für Normalized Discounted Cumulative Gain und wird im Kapitel 2 näher erläutert.

Die für die spätere Referenzkonfiguration A08 ausgewählten Modellparameter lauten:

$$n_estimators = 900 \tag{8.2}$$
$$learning_rate = 0.025 \tag{8.3}$$
$$num_leaves = 15 \tag{8.4}$$
$$min_child_samples = 50 \tag{8.5}$$
$$lambdarank_truncation_level = 5 \tag{8.6}$$

Zusätzlich wird eine Kandidatenprofilbreite von 16 Artikeln pro Makro verwendet.

Wie in Kapitel **??** beschrieben, werden Kandidaten einer Anfrage als gemeinsame Rankinggruppe betrachtet. Entsprechend entsteht für jede Anfrage eine Gruppe:

$$G_q = \{(q, c_1), (q, c_2), \ldots, (q, c_n)\} \tag{8.7}$$

mit der Anfrage q und den zugehörigen Makrokandidaten c_i.

Dadurch erlernt das Modell nicht die allgemeine Zugehörigkeit eines einzelnen Kandidaten zu einer Klasse. Stattdessen wird die optimale Kandidatenreihenfolge innerhalb derselben Anfrage gelernt. Ziel dabei ist es, dass der relevanteste Kandidat möglichst weit oben in der Rangliste erscheint.

Die Eingaben für das Modell sind die im Kapitel **??** beschriebenen Pairwise-Merkmale. Das Training erfolgt dabei ausschließlich auf den Trainingsdaten, während die Optimierung der Hyperparameter über die Validierungsdaten erfolgt (Pipeline step 931 2026).

8.7 Kapitelergebnis

Artefakt A4: Trainiertes Empfehlungssystem

9 Evaluierung

9.1 Evaluierungskonzept

Recall@k MRR nDCG

9.2 Versuchsaufbau

9.3 Versuchsergebnisse

A01-A09

Um verschiedene Modellkonfigurationen zu vergleichen, wurde die aktuelle Versuchsserie A01 bis A09 durchgeführt. Dabei wurden unterschiedliche Strategien, Profilbreiten und Regularisierungsparameter getestet. Ziel davon ist es, die Einflüsse einzelner Parameter auf die Modellleistung zu analysieren, (Alstetter 2026).

9.3.1 Untersuchte Konfigurationsparameter

Die verschiedenen Varianten der Konfiguration haben unterschiedliche Auswirkungen auf die Trainingslabels, die Kandidatenprofile sowie die Modellregularisierung. Die untersuchten Konfigurationsparameter werden in Tabelle 9.1 erläutert.

Tabelle 9.1: Untersuchte Konfigurationsparameter

Parameter	**Bedeutung**
Same-Family-Gewicht	Negative Kandidaten aus derselben Makrofamilie werden innerhalb der Verlustfunktion unterschiedlich gewichtet. Dadurch kann die Variantenauflösung stärker betont werden.
Profilbreite	Begrenzt die Anzahl der gespeicherten Profilwerte innerhalb eines Kandidatenprofils. Kleinere Profilbreiten erzeugen eine kompaktere Kandidatenrepräsentation.
L1-Regularisierung (α)	Bestraft große Modellgewichte und reduziert die Modellkomplexität. Dadurch werden irrelevante Merkmale stärker unterdrückt.
L2-Regularisierung (λ)	Stabilisiert die Gewichtung der einzelnen Merkmale und reduziert die Gefahr einer Überanpassung an einzelne Trainingsbeispiele.
Min. Split Gain	Definiert den minimal notwendigen Informationsgewinn, um eine Baumaufteilung im LightGBM-Modell zu ermöglichen. Höhere Werte führen dadurch zu konservativeren Entscheidungsbäumen.

Bei der L1-Regularisierung wird die Summe der Absolutwerte der Modellgewichte bestraft:

$$\lambda_1 \sum_j |w_j|. \tag{9.1}$$

Dadurch werden einzelne Gewichte auf null gesetzt. Die L2-Regularisierung bestraft dagegen die Summe der quadrierten Gewichte:

$$\lambda_2 \sum_j w_j^2. \tag{9.2}$$

Dadurch werden besonders große Gewichte begrenzt. Bei LightGBM beziehen sich diese Regularisierungsstrafen hauptsächlich auf die Ausgabewerte der Baumblätter.

A02 bis A07 wurden so gewählt, dass immer nur ein Parameter gezielt verändert wurde. Dadurch lassen sich die Auswirkungen der einzelnen Konfigurationsparameter isoliert untersuchen. Bei A08 und A09 werden mehrere Parameter (L1 und L2) gleichzeitig verändert, da diese bei LightGBM konzeptionell denselben Hebel darstellen (Bestrafung der Blattgewichte). Um eine stärkere Regularisierung zu erzielen, ist es in der Praxis sinnvoll, beide gemeinsam hochzudrehen, nicht nur einen isoliert. Bei A09 kommt zusätzlich ein positiver Wert für $\mathrm{min_split_gain}$ zum Einsatz, um dadurch einen noch größeren Einfluss auf eine stärkere Regularisierung zu erzielen, (Alstetter 2026).

9.3.2 Referenzkonfiguration A01

A01 dient als Referenzkonfiguration und bildet die Basis für die Vergleichsexperimente. Sie enthält die Standardwerte für alle untersuchten Konfigurationsparameter. Alle weiteren Konfigurationen A02 bis A09 werden gegenüber dieser bewertet.

Die Referenzparameter sind in Tabelle 9.2 zusammengefasst.

Tabelle 9.2: Referenzkonfiguration A01

Parameter	**Wert**
Exaktes Targettraining	Ja
Same-Family-Gewicht	5,0
Profilbreite	16
L1-Regularisierung (α)	0,05
L2-Regularisierung (λ)	0,2
Min. Split Gain	0,0

Ausgehend von dieser Basis werden die Varianten der Konfigurationen A02 bis A09 verändert. Tabelle 9.3 zeigt die gegenüber A01 vorgenommenen Veränderungen.

Tabelle 9.3: Abweichungen der Konfigurationen gegenüber der Referenz A01

Konfiguration	**Änderung gegenüber A01**
A02	Same-Family-Gewicht von 5,0 auf 2,0 reduziert
A03	Same-Family-Gewicht von 5,0 auf 8,0 erhöht
A04	Same-Family-Gewicht von 5,0 auf 12,0 erhöht
A05	Hierarchische Relevanzlabels statt ausschließlich exakter Ziel-Makros
A06	Profilbreite von 16 auf 12 reduziert
A07	Profilbreite von 16 auf 8 reduziert
A08	L1-Regularisierung auf 0,1 und L2-Regularisierung auf 0,5 erhöht
A09	L1-Regularisierung auf 0,5, L2-Regularisierung auf 1,0 und min_split_gain auf 0,01 erhöht

Dadurch lässt sich der Einfluss der jeweiligen Parameteränderung der entsprechenden Konfiguration zuordnen, (Alstetter 2026).

9.3.3 Ergebnisse

Tabelle 9.4 zeigt die getesteten Modellkonfigurationen und deren Leistung im Vergleich.

Tabelle 9.4: Ergebnisse der A01–A09-Konfigurationen auf dem Testdatensatz

Konfiguration	Recall@1	Recall@3	Recall@10	MRR
A01 Referenz	0,3241	0,4822	0,6798	0,4418
A02 Gewicht 2	0,3162	0,4862	0,6680	0,4399
A03 Gewicht 8	0,3241	0,4743	0,6838	0,4397
A04 Gewicht 12	0,2925	0,4783	0,6759	0,4195
A05 Hierarchisch	0,3162	0,4783	0,6719	0,4328
A06 Profilbreite 12	0,3162	0,4862	0,6877	0,4374
A07 Profilbreite 8	0,3241	0,4862	0,6719	0,4439
A08 Regularisierung mittel	**0,3478**	0,4783	0,6798	**0,4534**
A09 Regularisierung stark	0,3399	0,4783	**0,6917**	0,4505

9.4 Analyse

Stärken Schwächen Fehlerbilder

9.4.1 Same-Family-Gewicht

Mit den Konfigurationen A02 bis A04 wird der Einfluss des Same-Family-Gewichts untersucht. Die Ergebnisse zeigen, dass eine Verringerung des Gewichts auf 2 den Recall@1 auf 31,62 % senkt, während eine Erhöhung auf 8 keine Auswirkung zeigt. Die Gewichtung auf 12 führt zu einer deutlichen Verschlechterung des Recall@1 auf 29,25 %.

Daraus lässt sich schließen, dass ein zu hohes Same-Family-Gewicht die Lernstabilität negativ beeinflusst.

9.4.2 Hierarchische Labeldefinition

A05 untersucht die hierarchische Relevanzdefinition. Makros (EPLAN GmbH & Co. KG 2025) derselben Familie und desselben Präfixes werden dabei hierarchisch relevant betrachtet. Es zeigt sich jedoch keine Verbesserung gegenüber dem exakten Targettraining. Der Recall@1 sinkt von 32,41 % auf 31,62 % und die MRR von 44,18 % auf 43,28 %. Die dadurch entstandene reichhaltigere Zieldefinition hat somit keinen positiven Einfluss auf die Modellleistung, (Alstetter 2026).

9.4.3 Profilbreite

In den Konfigurationen A06 und A07 werden unterschiedliche Profilbreiten untersucht. Bei A06 beträgt diese zwölf, bei A07 acht. Die Erhöhung der Profilbreite führt zu einer Verschlechterung des Recall@1 von 32,41 % auf 31,62 % und der MRR von 44,18 % auf 43,74 %. Eine Verringerung der Profilbreite führt zu keiner nennenswerten Veränderung der Modellleistung. Die Ergebnisse zeigen, dass eine verringerte Profilbreite die Modellleistung nicht signifikant verbessert, (Alstetter 2026).

9.4.4 Regularisierung

Die deutlichsten Verbesserungen werden durch die Veränderung der Regularisierung erzielt. A08 verwendet eine mittlere L1- und L2-Regularisierung ($\alpha = 0{,}1$, $\lambda = 0{,}5$). Dadurch wird mit einem Recall@1 von 0,3478 der höchste Top-1-Wert aller untersuchten Konfigurationen erreicht. Auch der höchste MRR-Wert von 0,4534 wird durch A08 erreicht. Mit A09 wird die Regularisierung nochmals deutlich erhöht ($\alpha = 0{,}5$, $\lambda = 1{,}0$). Zusätzlich wird ein positiver Wert für $\mathrm{min_split_gain}$ gesetzt, um die Stabilität der Baumaufteilung zu erhöhen. Diese Konfiguration erreicht den höchsten Recall@10 von 0,6917 und die zweithöchste MRR von 0,4505.

Die L1/L2-Regularisierung wirkt sich bei LightGBM direkt auf die Blattgewichte aus. Sie dämpft diese bei geringer Aussagekraft auf null und unterdrückt dadurch das Rauschen der 2.381 Merkmale bei nur 984 Trainingsbeispielen, ohne das schwache Unterscheidungssignal zwischen Makrofamilien-Varianten zu verlieren. Bei A09 wird durch die stärkere Dämpfung auch dieses Signal mit reduziert, was die Top-1-Präzision verringert, dabei aber die Gesamtrangfolge robuster macht, (Alstetter 2026).

9.4.5 Top-Konfiguration

Die beste Konfiguration der Studie ist A08, da sie sowohl den höchsten Recall@1 als auch den höchsten MRR erreicht(Alstetter 2026).

Die wichtigsten Kennzahlen dieser Variante sind in Tabelle 9.5 dargestellt.

Tabelle 9.5: Ergebnisse der besten Konfiguration A08

Kennzahl	Wert
Recall@1	0,3478
Family Recall@1	0,8735
Recall@10	0,6798
Family Recall@10	0,9368
MRR	0,4534
Mittlerer Zielrang	23,96
90 %-Perzentil	61,6
Schlechtester Rang	360

Die Fehleranalyse dieser Konfiguration zeigt, dass 7,9 % aller Anfragen einer fachlich falschen Makrofamilie zugeordnet wurden. 52,6 % der Fehlklassifikationen erfolgen innerhalb der korrekten Makrofamilie. Die größte Herausforderung liegt damit weiterhin in der Variantenauflösung technisch ähnlicher Makrofamilien, (Alstetter 2026).

9.5 Kapitelergebnis

Artefakt A5: Leistungsbewertung

10 Diskussion

10.1 Nutzenbewertung

Suchzeit Wiederverwendung

10.2 Wirtschaftliche Betrachtung

Verbindung zu Kapitel 1.

10.3 Grenzen

11 Grenzen und Ausblick

11.1 Grenzen des aktuellen Systems

Die Ergebnisse der Evaluierung zeigen, dass die größte Einschränkung des aktuellen Systems in der Variantenauflösung technisch ähnlicher Makrofamilien liegt. Dies lässt sich auf die Struktur sowie die Eigenschaften der verfügbaren Datenbasis zurückführen.(Alstetter 2026)

Tabelle 11.1 fasst die zentralen Grenzen des Systems und die daraus abgeleiteten Ansatzpunkte zusammen.

Tabelle 11.1: Übersicht der zentralen Grenzen und Ansatzpunkte

Grenze	**Beobachteter Befund**	**Ansatzpunkt**
Geringer Support	302 Kandidatenprofile basieren nur auf einem Trainingsartikel	Erweiterung der Datenbasis für seltene Ziel-Makros
Variantenauflösung	Family Recall@1 liegt deutlich über Target Recall@1	Spezialisierte Modellierung innerhalb der Makrofamilien
Support-Bias	Häufige Ziel-Makros besitzen stabilere Kandidatenprofile	Ausgleich seltener Profile durch zusätzliche Beispiele und Profilpflege

11.1.1 Geringer Support von Ziel-Makros

Eine zentrale Einschränkung ergibt sich aus der Support-Verteilung der Kandidatenprofile. Von insgesamt 375 vorhandenen Kandidatenprofilen basieren 302 Profile auf nur einem Trainingsartikel. Der Median-Support liegt bei 1 und der Mittelwert der Artikel pro Ziel-Makro (EPLAN GmbH & Co. KG 2025) beträgt 2,62.

Ein einzelner Artikel ist dabei nicht in der Lage, einen ausreichenden Support für die technische Vielfalt innerhalb einer Makrofamilie zu bieten. Die daraus entstandenen Kandidatenprofile sind somit von nur einem Artikel abhängig.

11.1.2 Variantenauflösung innerhalb einer Makrofamilie

Die Resultate der Evaluierung zeigen, dass das Modell die korrekte Makrofamilie deutlich zuverlässiger erkennt als das exakte Ziel-Makro (EPLAN GmbH & Co. KG 2025). A08 als beste Konfiguration erzielt dabei einen Family Recall@1 von 87,35 %, wohingegen der Target Recall@1 lediglich 34,78 % beträgt.

Zudem zeigt die Fehleranalyse, dass in 133 Testfällen die korrekte Makrofamilie erkannt wird, jedoch nicht das exakte Ziel-Makro (EPLAN GmbH & Co. KG 2025). Die größte Herausforderung liegt somit in der fehlerfreien Unterscheidung zwischen sehr ähnlichen Varianten innerhalb derselben Makrofamilie (Alstetter 2026).

11.1.3 Support-Bias häufiger Ziel-Makros

Der Support-Bias bezeichnet einen Rankingvorteil von Ziel-Makros mit hohem Kandidaten-Support. Da die Kandidatenprofile dieser Ziel-Makros auf mehreren Trainingsartikeln beruhen, können dadurch breitere technische Merkmale abgedeckt werden und dadurch häufiger durch das Modell gefunden werden.

Die durchgeführten Fehleranalysen zeigen einen deutlichen Support-Bias. In 55,6 % der Fälle besitzt das fälschlich ausgewählte Ziel-Makro (EPLAN GmbH & Co. KG 2025) einen höheren Support als das korrekte Ziel-Makro (EPLAN GmbH & Co. KG 2025). Im Durchschnitt verfügt das falsch gewählte Ziel-Makro dabei über etwa 10,6 Artikel mehr als das korrekte Ziel-Makro (Alstetter 2026).

Häufige Ziel-Makros (EPLAN GmbH & Co. KG 2025) erzeugen dadurch stabilere Kandidatenprofile und bekommen somit einen strukturellen Vorteil gegenüber selteneren Varianten.

11.2 Kritische Reflexion

11.3 Interpretation der Ergebnisse

Nachdem die Modellauswahl abgeschlossen wurde, erfolgt die finale Evaluation des ausgewählten Modells auf dem unberührten Testsplit.

Tabelle 11.2 zeigt die erzielten Ergebnisse des Referenzlaufs A08 auf dem Testsplit.

Tabelle 11.2: Ergebnisse des Referenzlaufs A08 auf dem Testsplit

Metrik	Wert
Target Recall@1	0,3478
Target Recall@3	0,4783
Target Recall@5	0,5494
Target Recall@10	0,6798
Family Recall@1	0,8735
Family Recall@10	0,9368
MRR	0,4534
Hierarchical nDCG@10	0,7449

Beim Testsplit zeigt sich ein ähnliches Bild wie beim Validierungssplit. Das Modell sagt bei 34,78 % der Anfragen das korrekte Ziel-Makro (EPLAN GmbH & Co. KG 2025) auf Platz 1 voraus, während die korrekte Makrofamilie in 87,35 % der Anfragen auf die erste Position gesetzt wird.

Wie beim Validierungssplit ist auch hier der deutliche Abstand zwischen Family Recall@10 (93,68 %) und Target Recall@10 (67,98 %) zu erkennen.

Der erreichte MRR-Wert von 45,34 % sowie die hierarchische nDCG@10 von 74,49 % bestätigen zusätzlich die Erkenntnisse aus dem Validierungssplit, (Alstetter 2026).

11.4 Ursachenanalyse der Rankingfehler

Um die Modellleistung detailliert erfassen zu können, werden die Top-1-Ergebnisse des Testsplits analysiert. Dabei werden die fachlichen Beziehungen zum tatsächlichen Ziel-Makro untersucht.

Tabelle 11.3 zeigt die Verteilung der einzelnen Fehlerkategorien.

Tabelle 11.3: Top-1-Analyse der Vorhersagen auf dem Testdatensatz

Kategorie	Anzahl	Anteil
Exaktes Ziel-Makro (EPLAN GmbH & Co. KG 2025)	88	34,78 %
Gleiche Familie, falsches Target	133	52,57 %
Gleiches Familienpräfix	12	4,74 %
Andere Familie	20	7,91 %

Die Fehleranalyse bestätigt die Beobachtungen aus den Metriken. Von insgesamt 253 Testanfragen werden 133 Fälle einer gleichen Familie, aber einer falschen Variante zugeordnet, was 52,57 % entspricht. In 7,91 % der Fälle wird eine andere Familie vorhergesagt. Das zeigt, dass das Modell die grundlegenden technischen Zusammenhänge zwischen Artikeln und Makrofamilien zuverlässig

erlernt hat. Die Analyse zeigt, dass die zentrale Herausforderung die Identifikation des korrekten Ziel-Makros (EPLAN GmbH & Co. KG 2025) innerhalb der richtigen Makrofamilie ist. Dafür muss die Variantenauflösung innerhalb der Makrofamilie weiter verbessert werden. Die große Differenz zwischen dem Target Recall@1 und dem Family Recall@1 unterstreicht diese Herausforderung zusätzlich. Dies wird im Kapitel 11 genauer analysiert, (Alstetter 2026).

12 Fazit und Ausblick

12.1 RQ1 beantworten

12.2 RQ2 beantworten

12.3 RQ3 beantworten

12.4 RQ4 beantworten

12.5 RQ0 beantworten

Abbildungsverzeichnis

1.1 Elektromotor 2
1.2 Beispielartikel Elektromotor 3
1.3 Vision des Empfehlungssystems 6

2.1 Schematischer Aufbau eines mehrschichtigen neuronalen Netzes 8
2.2 Vereinfachte Darstellung von Embeddings im dreidimensionalen Embedding-Raum 9
2.3 Prinzipielle Funktionsweise eines Sentence Transformers. Semantisch ähnliche Texte werden auf nahe beieinanderliegende Embeddings abgebildet, während inhaltlich unterschiedliche Texte größere Abstände im Embedding-Raum aufweisen. 12
2.4 Funktionsweise des Triplet Loss. Der Anchor (A) wird dem positiven Beispiel (P) angenähert, während der Abstand zum negativen Beispiel (N) vergrößert wird. Die Margin m definiert dabei den Mindestabstand, um den das negative Beispiel weiter vom Anchor entfernt liegen muss als das positive Beispiel. 13
2.5 Vergleich der Learning-to-Rank-Ansätze. Pointwise bewertet Dokumente einzeln, Pairwise vergleicht Dokumentenpaare miteinander und Listwise optimiert die vollständige Rangliste direkt. 14
2.6 Prinzip von LambdaRank. Vergleiche, deren Vertauschung die Rangliste stark verändert, erhalten eine höhere Gewichtung als Vergleiche mit geringem Einfluss auf die Rangfolge (Burges u. a. 2006). 15
2.7 Beispiel für Recall@3. Da sich das relevante Dokument C innerhalb der ersten drei Rangpositionen befindet, gilt die Anfrage als erfolgreich und trägt positiv zum Recall@3-Wert bei. 16
2.8 Beispiel zur Berechnung des Mean Reciprocal Rank. Das erste relevante Makro befindet sich auf Rang 3, wodurch sich ein reziproker Rang von $1/3$ ergibt. Frühere relevante Treffer führen zu höheren MRR-Werten. 17
2.9 Beispiel für nDCG. Die Relevanz eines Ergebnisses wird unabhängig von seiner Rangposition betrachtet. Obwohl Dokument C die höchste Relevanz besitzt, erscheint es erst auf Rang 3. nDCG erkennt diese Fehlplatzierung und bewertet die Rangliste schlechter als eine ideal sortierte Rangliste. 18

3.1 Verzahnung von DSRM und ADR. Die DSRM bildet den übergeordneten Forschungsrahmen, während ADR als iterativer Entwicklungszyklus innerhalb der Phase „Design & Entwicklung des Artefakts“ eingesetzt wird. 21

4.1 Beispielhafte Struktur eines XML-Artikels für einen Elektromotor. Die hierarchische Organisation ermöglicht die Speicherung von Herstellerinformationen, technischen Informationen sowie Referenzen auf zugehörige Makros. 25
4.2 Beispielhafte Struktur eines Motor-Makros im .ema-Format. Das Makro beschreibt die technische Umsetzung eines Elektromotors im Schaltplan und enthält Informationen zu Anschlussstrukturen, Bezeichnungen und Makrovarianten. . . . 26
4.3 Beispiel einer n:1-Beziehung zwischen Artikeln und Makros. Mehrere Artikel verweisen über die Eigenschaft P_ARTICLE_MACRO auf dasselbe Makro. . . 26
4.4 Beispiel für vollständige und unvollständige Artikeldaten. Fehlende technische Merkmale oder fehlende Artikel-Makro-Zuordnungen verhindern eine zuverlässige Verwendung der Datensätze im Training und führen daher zu deren Entfernung. 28
4.5 Auswirkung redundanter Artikel- und Makrodaten. Mehrfach vorhandene Datensätze erhöhen künstlich die Gewichtung bestimmter technischer Informationen. Durch die Entfernung redundanter Daten wird diese Verzerrung reduziert. 29
4.6 Entfernung nicht nutzbarer Metadaten. Technisch relevante Merkmale bleiben erhalten, während Metadaten ohne technischen Informationsgehalt entfernt werden. 30
4.7 Beispiel für die Normalisierung technischer Einheiten. Unterschiedliche Schreibweisen derselben Spannung werden in eine einheitliche Darstellung überführt, wodurch die Vergleichbarkeit der Daten verbessert wird. 31
4.8 Beispiel für die Klassifikation technischer Spannungswerte. Mehrere technisch ähnliche Spannungswerte werden durch eine Klassifizierung zu gemeinsamen Spannungsklassen zugeordnet. Dadurch können ähnliche Geräte trotz leicht unterschiedlicher Spannungswerte gemeinsam betrachtet werden. 32
4.9 Erzeugung eines Artikelvektors aus technischen Informationen eines Motorartikels. Die extrahierten Merkmale werden zu einer gemeinsamen technischen Repräsentation zusammengeführt und dienen als Eingabe für das Empfehlungssystem. 34
4.10 Erstellung des Makrovektor. Aus dem .ema-Makro werden strukturelle Informationen extrahiert und in einen numerischen Vektor überführt, der die technische Struktur des Makros für das Empfehlungssystem beschreibt. 35
4.11 Erstellung des Lexikalvektors. Textuelle technische Informationen aus Artikel- und Makrodaten werden extrahiert und in einem separaten Vektor zusammengeführt. Der Lexikalvektor ergänzt die strukturierten Informationen des Artikel- und Makrovektors um technische Begriffe und Bezeichnungen. 36
4.12 Architektur der Datenanalyse und Trainingsvorbereitung. 38

Tabellenverzeichnis

1.1 Beispielhafte Wirtschaftlichkeitsbetrachtung des Empfehlungssystems 5

3.1 Zuordnung von Forschungsfragen, DSRM-Phasen, Artefakten und Kapiteln . . . 22

8.1 Dimensionen der erzeugten Rankingmatrizen 43
8.2 Variierte Hyperparameter der Grid-Search (Pipeline step 931 2026) 44

9.1 Untersuchte Konfigurationsparameter 48
9.2 Referenzkonfiguration A01 49
9.3 Abweichungen der Konfigurationen gegenüber der Referenz A01 49
9.4 Ergebnisse der A01–A09-Konfigurationen auf dem Testdatensatz 50
9.5 Ergebnisse der besten Konfiguration A08 52

11.1 Übersicht der zentralen Grenzen und Ansatzpunkte 54
11.2 Ergebnisse des Referenzlaufs A08 auf dem Testsplit 56
11.3 Top-1-Analyse der Vorhersagen auf dem Testdatensatz 56

Literaturverzeichnis

Alstetter, Andreas (2026). *LTR Configuration Test Series*. Eigene Untersuchung. Interne Konfigurationsanalyse der LTR-Modelle (siehe S. 47–52, 54–57).

Burges, Christopher J. C., Robert Ragno und Quoc Le (2006). „Learning to Rank with Nonsmooth Cost Functions“. In: *Advances in Neural Information Processing Systems* (siehe S. 14, 15).

EPLAN (2022). *Effizientes Erstellen von Schaltplänen: Symbole, Makros und Makro-Bibliotheken in der Elektrokonstruktion*. EPLAN (siehe S. 1, 2).

EPLAN (2023). *Artikelverwaltung: Grundlagen*. EPLAN (siehe S. 2).

EPLAN GmbH & Co. KG (2025). *Makros*. EPLAN-Plattform Infoportal (siehe S. 50, 54–57).

Järvelin, Kalervo und Jaana Kekäläinen (2002). „Cumulated Gain-Based Evaluation of IR Techniques“. In: *ACM TOIS*. Bd. 20. 4, S. 422–446. DOI: 10.1145/582415.582418 (siehe S. 17).

LeCun, Yann, Yoshua Bengio und Geoffrey Hinton (2015). „Deep Learning“. In: *Nature* 521.7553, S. 436–444 (siehe S. 7, 10).

Liu, Tie-Yan (2009). „Learning to Rank for Information Retrieval“. In: *Foundations and Trends in Information Retrieval* 3.3, S. 225–331. DOI: 10.1561/1500000016 (siehe S. 13).

Manning, Christopher D., Prabhakar Raghavan und Hinrich Schütze (2008). *Introduction to Information Retrieval*. Cambridge University Press (siehe S. 15).

Microsoft Corporation (2026). *LightGBM*. Microsoft Corporation (siehe S. 45).

Mikolov, Tomas, Wen-tau Yih und Geoffrey Zweig (2013). „Linguistic Regularities in Continuous Space Word Representations“. In: *Proceedings of NAACL-HLT*, S. 746–751 (siehe S. 8).

Organisation for Economic Co-operation and Development (OECD) (2024). *The OECD's Contribution to Policies to Optimise the Digital Transformation*. OECD (siehe S. 1).

Peffers, Ken, Tuure Tuunanen, Marcus A. Rothenberger und Samir Chatterjee (2007). „A Design Science Research Methodology for Information Systems Research“. In: *Journal of Management Information Systems* 24.3, S. 45–77. DOI: 10.2753/MIS0742-1222240302 (siehe S. 20).

Pipeline step 931 (2026). *Pipeline step 931: Learning-to-Rank macro ranker training*. Interner Projektquellcode (siehe S. 44–46).

Reimers, Nils und Iryna Gurevych (2019). „Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks“. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, S. 3982–3992. DOI: 10.18653/v1/D19-1410 (siehe S. 9, 12).

Schroff, Florian, Dmitry Kalenichenko und James Philbin (2015). „FaceNet: A Unified Embedding for Face Recognition and Clustering“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, S. 815–823. DOI: 10.1109/CVPR.2015.7298682 (siehe S. 13).

Sein, Maung K., Ola Henfridsson, Sandeep Purao, Matti Rossi und Rikard Lindgren (2011). „Action Design Research“. In: *MIS Quarterly* 35.1, S. 37–56. DOI: 10.2307/23043488 (siehe S. 20).

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser und Illia Polosukhin (2017). „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems* (siehe S. 11).

Voorhees, Ellen M. und Dawn M. Tice (1999). „The TREC-8 Question Answering Track Evaluation“. In: *Proceedings of the Eighth Text REtrieval Conference (TREC-8)*. DOI: 10.6028/NIST.SP.500-246.qa-overview (siehe S. 16).

Wei, Jason, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai und Quoc V. Le (2022). „Finetuned Language Models Are Zero-Shot Learners“. In: *International Conference on Learning Representations (ICLR)* (siehe S. 11).

World Economic Forum und Accenture (2026). *Intelligent Industrial Operations Outlook 2026*. World Economic Forum (siehe S. 1).

World Economic Forum und Boston Consulting Group (2023). *Harnessing the AI Revolution in Industrial Operations: A Guidebook*. World Economic Forum (siehe S. 1).

Zhang, Shengyu, Linfeng Dong, Xiaoya Li und et al. (2025). „Instruction Tuning for Large Language Models: A Survey“. In: *ACM Computing Surveys*. DOI: 10.1145/3777411 (siehe S. 11).

Zhao, Wayne Xin, Kun Zhou, Junyi Li und et al. (2026). „A Survey of Large Language Models“. In: *Frontiers of Computer Science*. DOI: 10.1007/s11704-026-60308-3 (siehe S. 10).

Zhu, Yutao, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou und Ji-Rong Wen (2024). „Large Language Models for Information Retrieval: A Survey“. In: *arXiv preprint arXiv:2308.07107* (siehe S. 16).